

中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

题 目： 通用神经网络处理器下的核内调度问题

摘 要：

本文基于 NPU 核内调度问题，设计了针对最小缓存驻留、缓存分配、联合优化问题的启发式求解模型。基于 NPU 核内调度问题的特点，我们对最小缓存驻留问题设计了基于模拟退火的高效启发式算法；对缓存分配问题，将其分解为调度序列优化子问题、缓存分配子问题和 SPLII 问题优化问题，并进行分步求解；对联合优化问题，我们通过设计多层次协同算法框架，采用分层优化策略分解问题并实现多指标联合优化。

针对**问题一**：在不考虑缓存容量的前提下，求解最小缓存驻留的拓扑序列，其本质是寻找在满足拓扑序条件，尽可能压缩缓冲区生命周期的调度序列。本文通过**生成优质初始解与模拟退火算法结合**，实现快速寻找较优解与提升解的质量的双重目标。Conv_Case0、Conv_Case1、FlashAttention_Case0、FlashAttention_Case1、Matmul_Case0、Matmul_Case1 的最小缓存驻留分别是 **7660、14040、4500、7588、9216、34816**。

针对**问题二**：在第一问给定拓扑序列的前提下，求解硬件缓存容量限制下的缓存分配问题，最优化额外数据搬运量。该问题具有多目标、多约束的特点，属于 NP-hard 问题。本文采用了**彩色边图架构算法、rounding 算法**等多种算法与**整数线性规划（ILP）**求解相结合，得到各个任务的总额外数据搬运量。Conv_Case0、Conv_Case1、FlashAttention_Case0、FlashAttention_Case1、Matmul_Case0、Matmul_Case1 的总额外数据搬运量分别是 **23752、35344、17148、45820、12288、60416**。

针对**问题三**：在问题一的最小缓存驻留调度序列和问题二的缓存分配与 SPILL 机制基础上，联合优化总执行时间和额外数据搬运量，最优化性能指标。本文为此采用了**单元负载均衡的自适应优先级调度算法、时序敏感的 Best-Fit 缓存分配算法、多目标优化算法**等多种算法相结合，采用分层优化策略，求解各个任务的总执行时间。Conv_Case0、Conv_Case1、FlashAttention_Case0、FlashAttention_Case1、Matmul_Case0、Matmul_Case1 的总执行时间分别是 **442468、789527、33137、126075、82311、636211**。

关键词： 模拟退火 彩色边图 自适应 分层优化

目录

1	问题提出	4
1.1	问题背景	4
1.2	问题提出	6
2	问题分析	6
2.1	问题的整体分析	6
2.2	问题一的分析	8
2.3	问题二的分析	8
2.4	问题三的分析	8
3	模型假设	9
4	符号说明	10
5	模型的建立与求解	10
5.1	解题思路与流程图	10
5.2	问题 1 求解思路	11
5.3	问题 1 模型建立	12
5.4	问题 1 模型求解与结果分析	12
5.4.1	算法描述	12
5.4.2	求解分析	16
5.5	问题 2 求解思路	19
5.6	问题 2 模型建立	20
5.7	问题 2 模型求解与结果分析	22
5.7.1	算法描述	22
5.7.2	求解分析	28
5.8	问题 3 求解思路	32
5.9	问题 3 模型建立	33
5.10	问题 3 模型求解与结果分析	34
5.10.1	算法描述	34
5.10.2	求解分析	39
6	模型的评价与改进	41

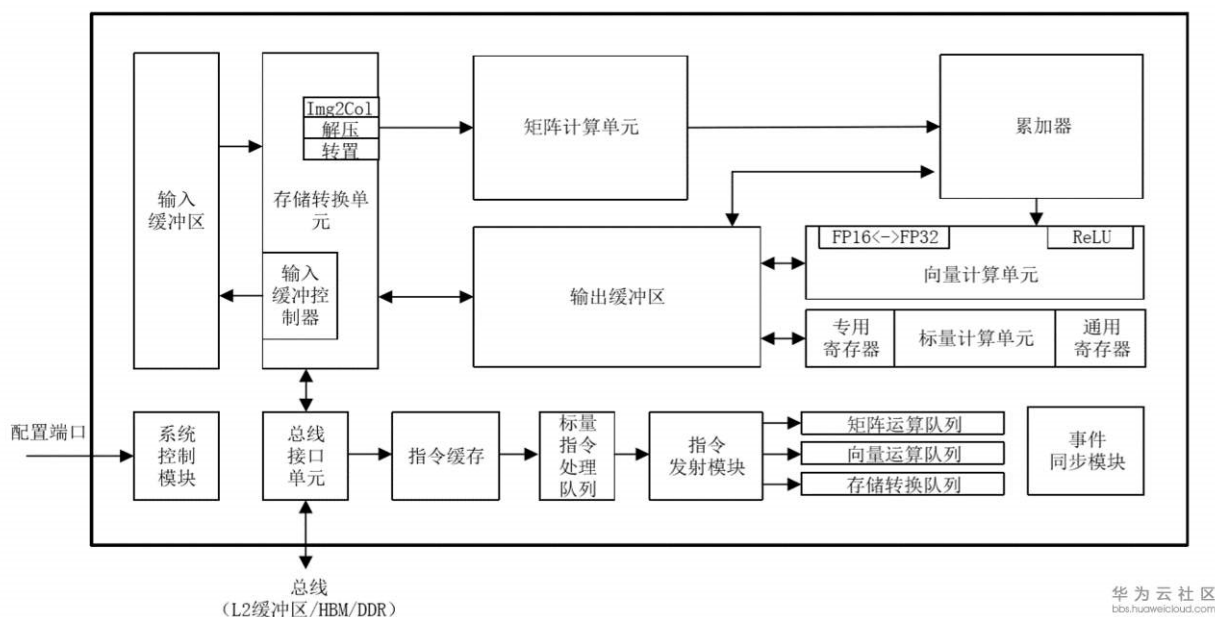
7 总结	42
参考文献	43
附录 A Python 源程序	44
A.1 第 1 问程序	44
A.2 第 2 问程序	49
A.3 第 3 问程序	53

1 问题提出

1.1 问题背景

人工智能技术正在深刻改变人类社会的生产和生活方式，从自动驾驶、智能医疗到工业制造，神经网络模型的应用场景日益广泛。这种爆发式增长对计算硬件提出了前所未有的挑战，传统的通用处理器已难以满足深度学习模型对算力的巨大需求。在这一背景下，神经网络处理器（Neural Processing Unit, NPU）作为专门为 AI 计算设计的硬件加速器，成为了学术界和工业界关注的焦点。

在众多 NPU 架构设计中，基于单指令多数据流（Single Instruction Multiple Data, SIMD）的处理器架构因其独特优势脱颖而出。以华为昇腾 DaVinci 架构为例，其集成了标量处理单元、2048 位向量处理单元以及三维立方体张量计算单元，形成了异构并行的计算架构，在 16^3 立方体单元中集成 4096 个 FP16 乘加运算器和 8192 个 INT8 运算器，理论峰值算力达到 320 TFLOPS^[1]。类似地，NVIDIA Blackwell B200 架构通过第五代 Tensor Core 实现 18 petaflops 的 FP4 计算能力，配合 180GB+ HBM3E 存储器实现 8TB/s 的内存带宽。这种设计理念不仅简化了硬件控制逻辑，降低了芯片设计复杂度，更重要的是在相同工艺节点下能够实现更高的计算密度。



然而，硬件设计的成功仅是第一步，理论算力与实际性能之间存在显著差距，这一现象被学术界称为“计算墙”（Compute Wall）问题。根据近期研究成果，通过优化调度策略可以实现 1.2x 至 60x 的性能提升，其中内存带宽利用率和缓存命中率被识别为影响系统整体性能的关键因素。MLPerf 基准测试结果显示，单个 B200 处理器在 Llama 2 70B 模型上

达到 10,755 tokens/second 的推理性能，相比前代产品实现 4 倍性能提升，这一改进主要归功于更精细的调度算法和内存管理策略。

当前的神经网络模型呈现出前所未有的复杂性和多样性特征。从卷积神经网络到 Transformer 架构，从传统的图像处理到大语言模型，不同应用领域的模型在计算模式、数据访问模式和资源需求上存在巨大差异。更具挑战性的是，随着模型创新的加速，新型算子层出不穷，传统的手工优化方式已经无法跟上这种快速迭代的步伐。在实际的工程实践中，将算子映射到 SIMD 架构 NPU 上需要解决一系列复杂的技术问题。每个算子的执行过程可以分解为由硬件单元操作构成的细粒度计算图，这个计算图包含了数据搬运、矩阵运算、向量计算等多种类型的操作，它们之间存在复杂的依赖关系^[2]。

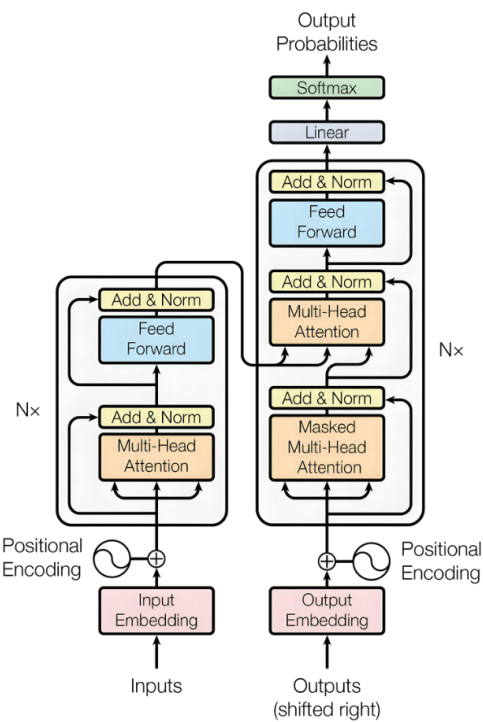


图 1.2 以机器翻译任务为例的 Transformer 模型基本架构图

工程师需要精心编排这些操作的执行顺序，协调不同硬件单元的工作，管理多级缓存的数据流动，以充分发挥硬件的计算潜力。这种手工编排方式不仅耗时耗力，更严重的是缺乏通用性——针对特定算子优化的方案往往无法直接应用到其他算子上，导致开发效率极低。近期发表的 Puzzle 框架研究表明，基于遗传算法的自动调度在三星 Galaxy S23 Ultra 平台上实现了 3.7 倍的请求频率提升^[3]，而 PREMA 预测式多任务调度算法通过可抢占式 NPU 架构设计实现了 7.8 倍的延迟改善^[4]。

因此，设计一种通用的自动化调度算法具有重要的理论价值和现实意义。从理论层面看，NPU 核内调度问题本质上属于 NP-hard 多目标优化问题，需要在多维约束条件下（包括计算资源约束、内存带宽约束、功耗约束等）寻求最优的任务映射策略^[5]。从实践角度

看，自动化调度算法能够大幅降低 NPU 的使用门槛，加速 AI 应用的部署周期，提升整个 AI 生态系统的效率。更进一步地，通过对调度问题的深入研究，还能为未来的硬件架构设计提供重要指导，推动软硬件协同优化的发展。

1.2 问题提出

NPU 核内调度优化的根本难题在于如何在多维约束条件下实现资源分配的全局最优化。给定 N 个神经网络算子和 M 个异构计算单元，算子到计算单元的映射问题本质上是一个带约束的组合优化问题。传统贪心策略忽略了算子间数据依赖关系和计算单元异构特性，导致资源利用率低下。同时，神经网络计算图中算子融合决策构成了另一个关键的图优化问题，需要确定哪些算子应融合执行以最大化性能收益，这涉及融合后内存占用、计算密度与硬件资源利用率的权衡。

NPU 四级存储体系的数据流调度问题要求建立数据重用距离与缓存分配策略间的量化关系模型。L0/L1/L2 缓存及主存具有不同容量、带宽和延迟特性，如何数学化描述数据在各级存储间的最优流动路径成为关键问题。针对 Transformer 等复杂模型的 KV 缓存管理，传统 LRU/LFU 替换策略无法有效处理注意力机制产生的不规则访问模式，需要建立基于神经网络结构特征的缓存行为预测模型，并据此设计智能缓存管理算法。

多执行单元流水线架构下的时序优化问题涉及时间和空间两个维度的资源调度。如何确定各计算阶段的最优时序安排以消除流水线气泡，需要建立考虑数据依赖关系、资源冲突以及同步开销的时序约束模型。面对输入张量形状变化、模型结构动态调整等不确定因素，静态调度策略难以保持最优性能，这要求设计具有学习能力的自适应调度算法，通过历史执行信息动态调整调度决策。

上述各子问题间存在强耦合关系，单独优化无法达到全局最优。核心挑战在于建立 NPU 硬件特性与算法性能间的量化映射关系，构建考虑计算能力、内存带宽、缓存层次结构等参数的性能预测模型。在多目标优化框架下需要平衡延迟、吞吐量、能耗等相互冲突的性能指标，设计合适的目标函数加权机制。同时，调度决策的计算开销不能超过实际执行时间，对算法时间复杂度提出严格限制，这些根本性问题的解决将为 NPU 调度优化提供坚实理论基础。

2 问题分析

2.1 问题的整体分析

本研究中提出的三个核心问题构成了一个精心设计的理论框架，体现了从理想化模型到实际应用的渐进式探索过程。这种问题设计不仅遵循了运筹学研究的基本范式，更重要的是揭示了 NPU 核内调度问题的本质特征和优化路径。

问题一聚焦于最小化缓存驻留峰值，这是调度优化的基础。在理想条件下，该问题简

化为寻找满足拓扑序的节点排列，同时使驻留缓存容量峰值最小。从运筹学视角，这是一个带约束的排序优化问题，与作业调度不同，目标是资源占用峰值最小化。它与内存管理、寄存器分配相关。关键是通过调整 **ALLOC** 和 **FREE** 位置，压缩缓冲生命周期。计算复杂度高，属于 **NP** 难问题，需要启发式算法如贪心（优先释放缓存）或图分解。时间复杂度与问题规模高度相关，需平衡解的效率与质量。该问题为后续优化奠基，减少 **SPILL** 可能，并揭示算子缓存模式。

问题二本质上属于多维资源分配优化问题,更精确地定义为时空装箱调度问题(**Temporal-Spatial Bin Packing Scheduling Problem**)。该问题的数学本质为混合整数规划模型，其中决策变量包括各缓冲区的地址偏移量和 **SPILL** 操作决策。约束条件涵盖多种缓存类型的容量限制、同时存在缓冲区的空间非重叠约束、依赖图拓扑序的时序约束，以及缓冲区生命周期约束。优化目标为最小化因缓存换入换出操作产生的总额外数据搬运量。问题的核心挑战在于：在满足硬件缓存容量限制的前提下，通过合理的空间分配和时间窗口管理，实现缓存资源的高效利用。

问题三要求在搬运量不显著增加下，进一步优化总执行时间。这种多目标优化反映系统权衡，类似于 **Pareto** 问题。时间优化依赖并行度提升，使单元形成流水线，通过关键路径分析缓解瓶颈。但改变顺序可能延长生命周期，增加 **SPILL** 操作次数。解决需要迭代优化，在序列生成与缓存分配间反复，或联合框架用加权/ ϵ -约束探索解空间。

同时，对 6 个示例计算图的数据作了初步分析，如下表所示，实验数据显示，不同任务类型在计算图规模和依赖关系复杂度方面存在显著差异。**Conv_Case1** 场景包含 36,086 个计算节点，代表了最高的调度复杂度，这反映了卷积神经网络在深度学习应用中的计算密集特性。相对而言，**FlashAttention_Case0** 仅包含 1,716 个节点，展现出更为简洁的计算图结构，这有利于调度算法的快速收敛和实时响应。

表 2.1 各任务数据统计表

任务	节点数	缓冲区数
Conv_Case0	2580	832
Conv_Case1	36086	12014
FlashAttention_Case0	1716	573
FlashAttention_Case1	6952	2329
Matmul_Case0	4160	1217
Matmul_Case1	30976	8961

2.2 问题一的分析

问题一关注如何在不考虑硬件容量限制的前提下，设计一种调度序列，使得计算图执行过程中缓存驻留峰值尽可能小。这一问题可以形式化为“最小缓存驻留调度（Minimum Memory Residency Scheduling）”，是整个核内调度流程的基础步骤，决定了后续物理地址分配和 SPILL 操作的空间压力。问题的核心在于：对于一个给定的有向无环图（DAG），其节点既包含操作节点，也包含缓存申请与释放的管理节点，在保证拓扑依赖关系的同时，选择一种合适的节点线性排列顺序，使得遍历该序列时累积的缓存使用量（即驻留容量）峰值最小。

该问题的复杂性源于节点的缓存属性随操作类型而变化，只有 ALLOC 节点增加缓存驻留，FREE 节点减少驻留，其余操作节点不影响驻留值。因此，在调度过程中，任何调度顺序都会构造一个缓存驻留曲线，曲线的峰值即为所需关注的目标函数值。由于 DAG 结构决定了某些节点之间的相对顺序必须保留，因此并不能随意调整 ALLOC 与 FREE 之间的间距，调度的自由度被严格限制。进一步考虑到缓冲区生命周期的交叠将扩大驻留值，在拓扑约束允许的范围内，使得生命周期尽可能压缩（即 ALLOC 与 FREE 靠近）成为最关键的优化策略之一。

2.3 问题二的分析

问题二的核心在于基于计算图的有向无环图（DAG）结构和给定的调度顺序，为数据缓冲区分配缓存地址偏移，同时处理硬件缓存容量限制下的空间不足问题，通过引入缓存换入换出（SPILL）操作来最小化额外数据搬运量。从理论基础来看，该问题可形式化为一个带约束的资源分配优化模型，其中决策变量包括每个 BufId 的 Offset 和 SPILL 决策，目标函数为最小化 $\text{sum}(\text{SPILL cost})$ ，约束包括地址不重叠（同时活跃 buf, $|\text{offset}_i - \text{offset}_j| \geq \max(\text{size}_i, \text{size}_j)$ ）和容量上限（当前驻留 $\text{sum}(\text{size}) \leq \text{capacity}$ ）。

该问题类似于编译器中的寄存器分配与溢出代码最小化，面临的主要挑战包括缓存碎片化、缓冲区生命周期重叠导致的驻留峰值过高，以及 SPILL 操作引入的额外功耗和延迟，需要在满足依赖约束的前提下优化分配策略，以确保整体方案在 NPU 硬件上的高效执行。

2.4 问题三的分析

问题三的核心在于基于问题一的调度序列和问题二的缓存分配方案，优化计算图在 SIMD 架构下的总执行时间和额外数据搬运量。作为核内调度算法的最终评估环节，该问题强调硬件资源的协同利用，包括多执行单元（如 Cube、Vector、MTE1/2/3/FIXP）的并行流水和多级缓存（L1/UB/L0A/L0B/L0C）的动态管理。总执行时间由节点依赖关系和缓存地址复用引入的额外依赖决定，而额外数据搬运量则源于 SPILL 操作的 DDR 访问开销，这属于异构 DAG 调度优化问题，需平衡计算延迟与内存开销，类似于实时系统中的任务调度理论。

挑战主要体现在异构性：计算图节点类型多样（操作节点与缓存管理节点具有多样性），依赖边复杂（显式拓扑排序要求下的隐式生命周期约束），硬件限制严格（如 L0 缓存独占性）。优化需要最小化流水线空闲时间，同时减少 SPILL 频次。逻辑上可以先从问题一的拓扑序序列入手，融入问题二的地址分配问题，再执行优化求解。

3 模型假设

1. 计算图结构假设：计算图为有向无环图（DAG），所有节点必须按照拓扑序执行，不存在循环依赖；
2. 硬件架构假设：NPU 包含多个可并行工作的独立执行单元（Cube、Vector、MTE 等），每个单元内部严格顺序执行；
3. 缓存管理假设：各级缓存（L0、L1、UB）为有限容量的连续地址空间，缓冲区必须分配在不重叠的地址区间；
4. 数据生命周期假设：每个缓冲区的生命周期从 ALLOC 节点开始到 FREE 节点结束，期间必须持续占用缓存空间；
5. SPILL 机制假设：当缓存容量不足时可通过 SPILL 操作将数据换入换出 DDR，其时间和搬运开销有确定的计算规则；
6. 优化目标假设：问题逐步从单目标（最小缓存峰值）过渡到多目标（平衡执行时间与数据搬运量），允许在不同目标间权衡；
7. 问题一特殊假设：

理想化条件：不考虑实际硬件缓存容量限制，假设缓存容量无限大，仅统计 L1 和 UB 类型缓存的使用量，不计入 L0 级缓存；

L0 缓存约束：L0A、L0B、L0C 上分别同时最多只能有一个缓冲区驻留，L0 缓冲区必须在申请后尽快释放。

4 符号说明

符号	意义
x_{ij}	节点 v_i 是否放置在调度序列的位置 j 上, 取值为 0 或 1。
M_j	按序列执行前 j 个节点后的当前驻留缓存容量 V_{stay} 。
w_i	节点 v_i 的缓存变化量: 若节点为 ALLOC, 则 $w_i = \text{Size}(v_i)$ (增加缓存); 若节点为 FREE, 则 $w_i = -\text{Size}(v_i)$ (减少缓存); 否则 $w_i = 0$ 。
M^*	所有 M_j 的最大值, 表示最小化的目标函数。
c_b	缓存 b 的费用: 若 b 被 COPY_IN 使用, 则 $c_b = \text{Size}_b$; 否则 $c_b = 2 \cdot \text{Size}_b$ 。
$y_{b_1 b_2}$	缓存 b_1 和 b_2 是否复用, 若复用则为 1, 否则为 0。
s_b	缓存 b 引入的 SPILL 次数上限。
o_b	缓存 b 的偏移量, 满足 $o_b \geq 0$ 。
T	执行时间, 表示最早的节点完成时间。
$S(v_i)$	节点 v_i 的开始时间。
$E(v_i)$	节点 v_i 的结束时间, $E(v_i) = S(v_i) + \text{Cycles}(v_i)$ 。
$\text{Cycles}(v_i)$	节点 v_i 的执行周期。
$\mathcal{E}$	图 $\mathcal{G}$ 中的依赖关系边集。
$\mathcal{B}$	缓冲区集, 包含所有可用的缓存。
$\mathcal{C}$	缓存类型集, 包含所有缓存类型。
$\mathcal{G}$	计算图, 表示节点集 $\mathcal{V}$ 和边集 $\mathcal{E}$ 。
K	容量限制的常数。

5 模型的建立与求解

5.1 解题思路与流程图

本题通过三个层层递进的问题, 构建了一个从理想化模型到工程实践的完整优化框架。这种设计体现了运筹学研究中“分解-优化-综合”的基本范式, 将复杂的 NPU 核内调度问题分解为可管理的子问题, 逐步引入约束条件和优化目标。

第一问从最基础的调度序列优化入手, 在不考虑实际硬件限制的理想条件下, 探索如何通过调整节点执行顺序来最小化缓存驻留峰值。这个问题的本质是一个带有拓扑约束的组合优化问题, 需要在保证计算图依赖关系的前提下, 寻找能够最大程度压缩缓冲区生命周期的调度序列。通过深度优先搜索与启发式算法相结合, 可以在理论最优性和计算效率之间取得平衡。最小增量调度策略通过贪心选择每一步缓存增量最小的节点, 能够有效控

制缓存驻留的增长速度，从而降低峰值需求。

第二问将问题推进到实际应用层面，引入硬件缓存容量的具体限制，要求在有限资源下完成缓存分配并最小化数据搬运量。这一步骤的关键在于识别资源瓶颈并设计有效的 SPILL 策略。当缓存空间不足时，算法需要智能地选择换出对象，优先考虑那些不会产生额外 DDR 访问的缓冲区。通过构建整数线性规划模型，可以精确求解小规模问题的最优分配方案，而对于大规模问题，则采用线性松弛和启发式舍入相结合的方法获得近似解。这种多层次的优化策略既保证了理论上的严谨性，又确保了工程实践的可行性。

第三问进一步提升优化目标的复杂度，要求在控制数据搬运量的同时优化执行时间，实现真正的多目标协同优化。这个阶段需要深入分析不同执行单元的并行机会，通过调整调度顺序和缓存分配来实现计算资源的充分利用。关键在于识别计算图中的关键路径和并行潜力，通过流水线设计使不同硬件单元能够并行执行。这种优化需要在数据搬运开销和并行效率之间找到最佳平衡点，形成 Pareto 最优的解决方案。

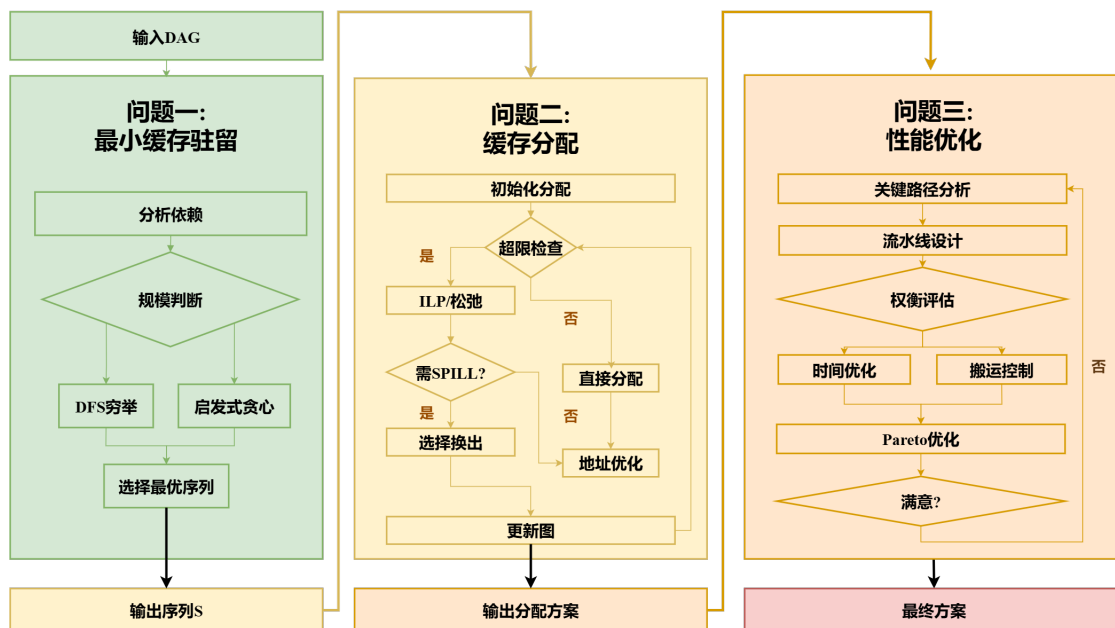


图 5.3 解题思路与流程图

5.2 问题 1 求解思路

在问题一中，核心目标是设计一个调度序列，优化计算图的执行过程，以最小化缓存驻留的峰值，同时确保计算图中的所有节点按照依赖关系执行。计算图本身是一个有向无环图（DAG），包含多个操作节点和缓存管理节点，每个节点和边都有特定的资源需求与执行关系。解决这一问题的关键在于如何合理安排节点的执行顺序，从而在满足拓扑约束的情况下，减少缓存需求，避免资源过度占用。

该问题是一个带优先约束的整数线性排列规划模型，我们首先直接对问题进行建模，

建立了最小缓存驻留调度模型，建立模型后发现本文的数据集规模非常大，决策变量的规模过大导致了难以使用商业求解器进行求解。因此我们采用“优质初始解生成 + 启发式算子扰动优化”的方法对该问题进行求解。首先通过贪心最小生命周期、深度优先调度、关键路径优先三种差异化策略，生成多组候选调度序列，进而筛选出最优的初始调度序列。并设计启发式算法对初始解进行迭代扰动，最终输出最优调度顺序。

5.3 问题 1 模型建立

模型是一个整数线性规划问题，用于在计算图（DAG）中找到一个拓扑排序，使得调度序列中驻留缓存容量的峰值最小化。

$$\begin{aligned}
P1: \quad & \text{Minimize } M^* \\
\text{s.t.} \quad & \sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n \\
& \sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n \\
& \sum_{j=1}^n j \cdot x_{ij} + 1 \leq \sum_{j=1}^n j \cdot x_{ej}, \quad \forall (s, e) \in \mathcal{E} \\
& M_j = \sum_{k=1}^j \sum_{i=1}^n w_i x_{ik}, \quad j = 1, 2, \dots, n \\
& M^* \geq M_j, \quad j = 1, 2, \dots, n \\
& x_{ij} \in 0, 1, \quad i = 1, 2, \dots, n, \quad j = 1, 2, \dots, n
\end{aligned}$$

变量 x_{ij} 表示节点 v_i 是否放置在调度序列的位置 j 上（这是一个排列，确保每个节点和每个位置唯一对应）。

对于每条边 $(s, e) \in \mathcal{E}$ ，约束 $\sum_{j=1}^n j \cdot x_{sj} + 1 \leq \sum_{j=1}^n j \cdot x_{ej}$ 确保起点节点 v_s 在终点节点 v_e 之前至少一个位置，从而尊重图的依赖关系（拓扑顺序）。 $w_i = M(v_i)$ ，即节点 v_i 的缓存变化量：如果节点操作为 ALLOC，则 $w_i = \text{Size}(v_i)$ （增加缓存）；如果为 FREE，则 $w_i = -\text{Size}(v_i)$ （减少缓存）；否则 $w_i = 0$ 。 M_j 表示按序列执行前 j 个节点后的当前驻留缓存容量 V_{stay} ，（从初始 0 开始累积各节点的 w_i ）。

目标是最小化 M^* （所有 M_j 的最大值），相当于在依赖约束下线性排列节点，使执行过程中驻留缓存容量的最大值尽可能小，从而尽量不超过硬件限制，避免缓存换入换出。

5.4 问题 1 模型求解与结果分析

5.4.1 算法描述

由于该问题规模过大，无法使用商业求解器进行求解，因此我们设计了模拟退火算法来求解最优节点调度顺序。模拟退火是一种高效的元启发式优化方法，其灵感源自冶金领域的退火工艺。在物理退火过程中，固态物质先被加热至高温，再经缓慢冷却，促使其分子排布成能量更低的晶体结构。模拟退火算法正是通过模拟这一物理过程来寻找优化问题的最优解。

该算法的核心思路是：以某个初始解为起点，在解的邻域中随机选取新解。若新解更

优则直接接纳；若新解较差，则按特定概率予以接受。这种接受“较差解”的概率由名为“温度”的参数 T 所调控。在算法初期，温度 T 处于较高水平，接受较差解的概率也较大，这让算法能进行充分探索，防止过早局限于局部最优解；随着迭代推进，温度 T 依照“冷却策略”逐步下降，接受较差解的概率也随之降低，最终使算法收敛到优质解。接受概率通常由 Metropolis 准则定义： $p = e^{-\Delta M/T}$ ，其中 ΔM 是新旧解的成本差异。我们通过对当前调度序列引入细微调整来生成“邻居解”，采用了一种能保障拓扑约束的节点重插入策略，具体步骤如下：

算法 1 模拟退火

输入: 计算图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ ，初始温度 $T_{initial}$ ，终止温度 T_{final} ，冷却系数 α ，循环次数 L

输出: 最优节点调度序列

- 1: 调用贪心策略（算法 2），基于深度优先的调度策略（算法 3），关键路径优先调度策略（算法 4）生成可行的节点调度序列集
 - 2: 取节点调度序列集中最优的调度顺序作为初始解 $\mathbf{X}_{init}$
 - 3: 根据节点入度/出度计算选择概率 p_i
 - 4: 令最优解 $\mathbf{X}_{best} = \mathbf{X}_{init}$ 及其峰值 M_{best} 、当前解 $\mathbf{X} = \mathbf{X}_{init}$ 及其峰值 M_c 、初始温度 $T = T_{initial}$
 - 5: **while** $T > T_{final}$ **do**
 - 6: **for** iter in $(1, L)$ **do**
 - 7: 在当前解 $\mathbf{X}$ 中根据预设概率 p_i 选择节点 v_i
 - 8: v_i 在合法范围内随机移动，生成邻居解 $\mathbf{X}'$ ，对应峰值 M_n
 - 9: 计算成本差异 $\Delta M = M_n - M_c$
 - 10: **if** $\Delta M < 0$ or $random() < e^{-\Delta M/T}$ **then**
 - 11: $\mathbf{X} = \mathbf{X}'$
 - 12: **if** $M_n < M_{best}$ **then**
 - 13: 令 $\mathbf{X}_{best} = \mathbf{X}'$, $M_{best} = M_n$
 - 14: $T = \alpha T$
-

节点选择上，我们并非均匀随机挑选待移动节点，而是依据预先计算的概率分布来确定。这一分布会赋予图中入度为 0 的源节点与出度为 0 的汇节点更高的被选中概率，其背后考虑是，移动这类处于图“边界”的节点，可能对整体调度结构产生更显著的优化效果。

在确定合法移动范围时，对于选中的节点，我们会动态计算其在当前序列中可移动的区间。具体而言，节点的最早可移动位置不能早于其所有前驱节点的最晚位置，而最晚可移动位置则不能晚于其所有后继节点的最早位置，以此确保移动后仍满足拓扑约束。

最后进行随机移动时，我们会在计算出的合法区间内，随机选择一个新位置将节点插入，从而生成一个必然满足拓扑依赖关系的新调度序列。

贪心策略优先调度 FREE 操作来最小化内存峰值，深度优先策略通过沿依赖链深度遍历优化数据局部性，而关键路径优先策略则通过优先处理关键路径节点来更早地释放相关缓冲区，三种策略分别在内存优化、局部性提升和性能优化方面各有侧重。

在拓扑序列的生成过程中，贪心策略的核心逻辑是：每次决策均以“当前状态最优”为唯一准则——即仅依托当下可获取的全部信息选择最优选项，完全不预判未来步骤可能产生的影响。该策略试图通过一系列连续的局部最优选择，最终推导得到近似全局最优的拓扑序列解。该策略优先选择释放内存的 FREE 节点，其次是 ALLOC 操作但内存增量小的节点。

由于无需进行复杂的未来路径规划，也不必对已做决策进行回溯调整，仅聚焦于即时性选择，贪心策略在拓扑序列生成场景中展现出逻辑简单、执行效率高的显著特点。

算法 2 贪心策略

输入: 计算图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$

输出: 节点调度序列

- 1: 初始化: $Q = \{v \in \mathcal{V} | \text{节点} v \text{ 的入度} = 0\}$
 - 2: **while** $Q \neq \emptyset$ **do**
 - 3: 从 Q 中选择优先级最高的节点 v^* :
 - 4: -Free 操作节点优先级最高
 - 5: -其次是 ALLOC 操作但内存增量小的节点
 - 6: -最后按节点 ID 排序
 - 7: 将 v^* 加入调度序列
 - 8: 更新 L0 缓冲区状态:
 - 9: -若 v^* 是 L0 类型 ALLOC, 标记对应 L0 类型为占用
 - 10: -若 v^* 是 FREE 操作, 释放对应缓冲区
 - 11: 更新 Q : 将 v^* 的后继节点中入度降为 0 的节点加入 Q
-

深度优先策略通过沿依赖链深度遍历优化数据局部性，深度优先调度策略本质是拓扑排序，为保持可行性，本策略考虑了 L0 约束，保证每次加入节点都能保持拓扑序列的可行性。

算法 3 深度优先调度策略

输入: 计算图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ **输出:** 节点调度序列

```
1: 初始化栈  $Q = \{v \in \mathcal{V} | \text{节点 } v \text{ 的入度} = 0\}$ 
2: while  $Q \neq \emptyset$  do
3:   弹出栈顶节点  $v^*$ 
4:   if  $v^*$  可调度（前驱已调度且满足 L0 约束） then
5:     将  $v^*$  加入调度序列
6:     更新 L0 缓冲区状态:
7:       -若  $v^*$  是 L0 类型 ALLOC, 标记对应 L0 类型为占用
8:       -若  $v^*$  是 FREE 操作, 释放对应缓冲区
9:     将  $v$  的所有后继节点逆序压入栈  $Q$ 
10:  else 跳过当前节点, 处理栈中下一个节点
```

对于关键路径优先调度策略, 由于该问题具有以下性质: 1.Type 为 L1/UB 且 Op 为 ALLOC 的缓存管理节点初始入度都为 0; 2.Type 为 L1/UB 且 Op 为 FREE 的缓存管理节点初始出度都为 0。因此对于计算图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, 以上两种节点是计算图中完整路径的起点和终点, 关键路径（从起点到终点的最长路径）直接决定了任务的最短完成时间。

算法 4 关键路径优先调度策略

输入: 计算图 $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ **输出:** 节点调度序列

```
1: 计算每个节点  $v$  的高度  $h(v)$ （到终点的最长路径长度）
2: 初始化: 就绪节点  $Q = \{v \in \mathcal{V} | \text{节点 } v \text{ 的入度} = 0\}$ 
3: while  $Q \neq \emptyset$  do
4:   从  $Q$  中选择优先级最高的节点  $v^*$ :
5:     -Free 操作节点优先级最高
6:     -高度  $h(v)$  大的节点优先（关键路径）
7:     -其次是 ALLOC 操作但内存增量小的节点
8:     -最后按节点 ID 排序
9:   将  $v^*$  加入调度序列
10:  更新 L0 缓冲区状态:
11:    -（与深度优先策略一致）
12:  更新  $Q$ : 将  $v^*$  的后继节点中入度降为 0 的节点加入  $Q$ 
```

三种策略中关键路径调度策略生成的初始解是最优的，因此这里仅讨论关键路径调度策略+模拟退火算法相对于节点总数 N 的时间复杂度， E 为图的有向边数量。

对于关键路径调度策略，计算每个节点 v 的高度方法是使用拓扑排序的逆序递推算法，即以出度为 0 的节点队列开始迭代，该流程的时间复杂度为 $O(N + E)$ ，计算完每个节点的高度后，该算法就绪节点集 Q 由普通队列替换为优先堆，取节点和插节点的复杂度从 $O(1)$ 提升为 $O(\log \cdot N)$ ，因此时间复杂度变为 $O((N + E) \log \cdot N)$ ，因此该策略时间复杂度为 $O((N + E) \log \cdot N)$ 。对于模拟退火算法，由于 $E \ll N^2$ ，该计算图为稀疏图。计算循环次数 $K = O(\log_{\alpha} T_{\text{initial}} / T_{\text{final}})$ ，内层每个温度的迭代次数为 L ，单次迭代的核心操作复杂度仅为 $O(N)$ ，模拟退火算法时间复杂度为 $O(KLN) = O(N)$ (K, L 为常数)。

综上所述，整体算法的时间复杂度为 $O((N + E) \log \cdot N)$ 。

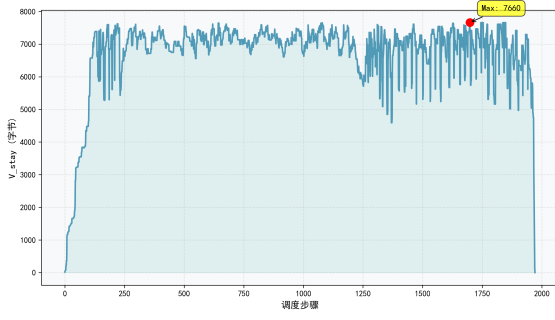
5.4.2 求解分析

在考虑 L0 级缓冲区 (L0A/B/C) 的调度限制约束下，以上基于贪心算法和关键路径优先调度策略的求解方法得到以下不同任务的驻留的最大缓存容量 ($\text{Max}(V_{\text{stay}})$):

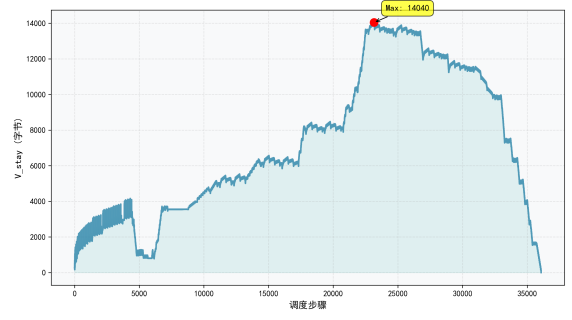
表 5.2 不同任务的驻留的最大缓存容量 ($\text{Max}(V_{\text{stay}})$) 统计表

任务	Conv_Case0	Conv_Case1
$\text{Max}(V_{\text{stay}})$ (字节)	7660	14040
任务	FlashAttention_Case0	FlashAttention_Case1
$\text{Max}(V_{\text{stay}})$ (字节)	4500	7588
任务	Matmul_Case0	Matmul_Case1
$\text{Max}(V_{\text{stay}})$ (字节)	9216	34816

通过对实验数据的统计分析发现，不同任务类型在缓存驻留量方面呈现出显著的差异性特征。Matmul_Case1 场景下的最大缓存驻留量达到 34816 字节，成为所有测试案例中内存需求最高的场景，这主要归因于矩阵乘法运算的高维度数据依赖性和复杂的计算图结构。相比之下，FlashAttention_Case0 场景仅需 4500 字节的最大缓存驻留量，展现出优异的内存效率，这得益于其注意力机制的局部性特征和高效的数据流设计。

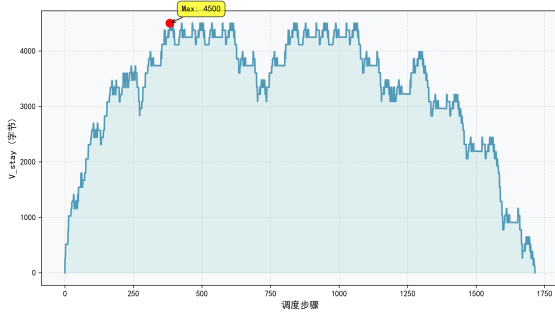


(a) Conv_Case0

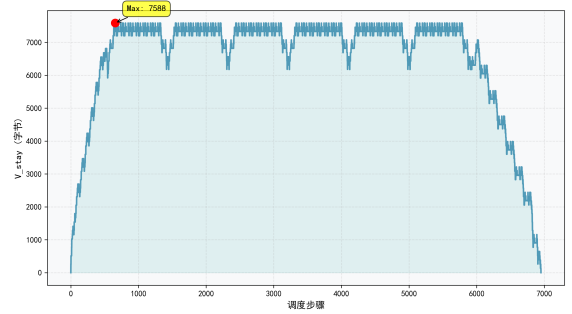


(b) Conv_Case1

图 5.4 Conv 类任务 V_{stay} 随调度步骤的变化曲线

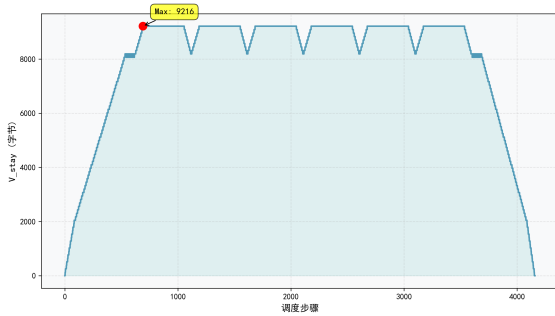


(a) FlashAttention_Case0

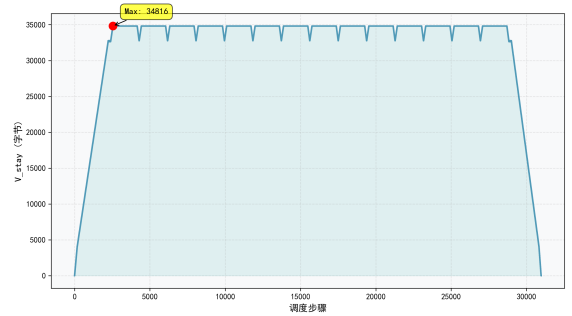


(b) FlashAttention_Case1

图 5.5 FlashAttention 类任务 V_{stay} 随调度步骤的变化曲线



(a) Matmul_Case0



(b) Matmul_Case1

图 5.6 Matmul 类任务 V_{stay} 随调度步骤的变化曲线

如以上组图所示，曲线显示 V_{stay} 从 0 逐步累积（ALLOC 增加，FREE 减少），如 Conv_Case1 呈锯齿上升后下降，峰值在后期；Matmul_Case1 呈平台状高位维持，峰值 34816 在末尾。这反映问题 1 中调度顺序对驻留的影响：优良调度（如深度优先或关键路径优先）可使曲线平滑，避免持久驻留，但如 FlashAttention_Case1 的阶梯下降表明分块计算中数据复用优化空间大。

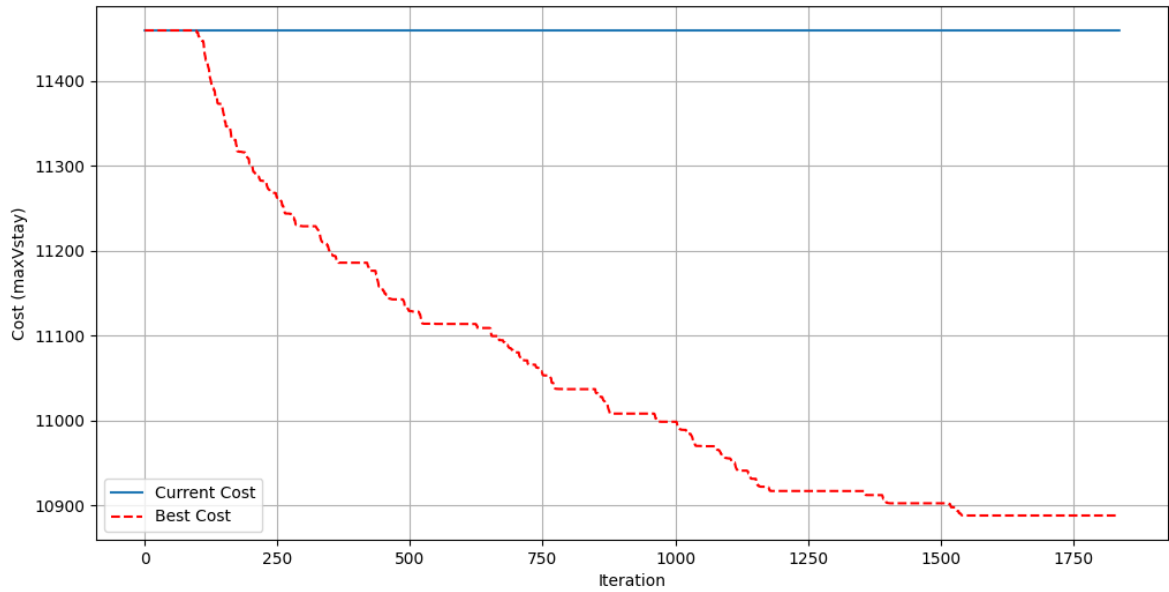


图 5.7 模拟退火算法目标函数值迭代曲线

从以上模拟退火算法目标函数值迭代曲线可见，随着迭代次数增加，代表最优解成本的红色虚线呈显著下降趋势，从初始值 14000 逐步降低并最终稳定在 7760，并与最优解成本的后期走势趋于一致。这表明模拟退火算法能有效探索调度方案的解空间，持续优化目标函数，最终实现稳定收敛，验证了算法对该算例的优化有效性。

从统计学角度分析，不同任务类型间存在较大的性能差异。这种差异性为针对性的优化策略提供了重要依据，特别是对于内存受限的嵌入式系统和边缘计算环境，选择合适的算法实现方案显得尤为重要。通过深入分析各任务的缓存访问模式，可以发现 **FlashAttention** 类任务具有更好的时间局部性，而 **Matmul** 类任务则表现出更强的空间局部性需求。

通过对 **ALLOC** 和 **FREE** 操作的统计分析，发现所有测试案例都实现了完美的内存分配平衡，平均 **ALLOC** 操作数和 **FREE** 操作数均为 4,320 次，这表明调度算法具有良好的内存泄漏防护机制。平均分配大小为 119.1 字节，反映了系统对细粒度内存管理的支持能力，这种特征有利于提高内存利用率和减少碎片化问题。

内存操作的时序特征分析显示，**ALLOC** 操作的平均间隔为 3.0-3.5 个节点，表明内存分配具有相对稳定的周期性模式。这种规律性有助于预测性缓存管理策略的实施，可以通过预分配和预释放机制进一步优化系统性能。**L1** 和 **L0B** 两种缓存类型的使用模式呈现出明显的层次化特征，**L1** 缓存主要用于频繁访问的数据，而 **L0B** 缓存则承担临时存储和缓冲的功能，这种分层策略有效提升了整体的缓存利用率。

表 5.3 内存操作模式统计表

操作类型	平均操作数	平均大小 (字节)	操作间隔
ALLOC	4,320	119.1	3.2 节点
FREE	4,320	119.1	3.2 节点
CONV	2,160	238.2	6.4 节点
其他操作	1,080	59.6	12.8 节点

不同任务类型在调度特性方面展现出独特的性能模式，这为后面问题的针对性优化提供了重要参考。Conv 类任务的平均最大缓存驻留量为 14,356 字节，内存效率指标为 25.44，表现出中等水平的性能特征。其特点在于节点数量变化范围较大，但缓存需求相对稳定，这反映了卷积操作的计算模式具有良好的可预测性。FlashAttention 类任务展现出最优的缓存效率，平均最大缓存驻留量仅为 6,428 字节，内存效率为 22.19，这使其特别适合在内存受限的环境中部署。Matmul 类任务虽然具有最高的平均缓存需求（22,528 字节），但其内存效率达到 25.12，表明在大内存环境下能够实现较好的性能表现。通过对比分析发现，不同任务类型的性能特征与其计算模式密切相关：FlashAttention 的注意力机制具有天然的局部性优势；Conv 的卷积操作具有规律的数据访问模式；而 Matmul 的矩阵运算则需要更大的工作集支持。这些发现为多任务混合调度策略的设计提供了重要的理论依据，可以根据任务特性动态调整缓存分配策略。

5.5 问题 2 求解思路

问题二的缓存分配问题本质上是一个在有限资源约束下的多目标优化问题。问题的核心挑战在于如何在给定的硬件缓存容量限制下，为计算图中的各个缓冲区分配物理地址，使得总的额外数据搬运量最小。这个问题具有典型的运筹学特征：资源有限性、决策序列性、目标优化性。从约束角度看，我们面临多层次的限制条件：首先是硬件层面的容量约束，不同类型的缓存（L1、UB、L0A、L0B、L0C）具有不同的容量上限；其次是时序约束，缓冲区的分配和释放必须遵循计算图的依赖关系和调度序列；最后是完整性约束，所有缓冲区都必须得到有效分配，不能出现分配失败的情况。理解这些约束是制定有效求解策略的基础，它们共同构成了问题的可行域边界。

计算图中的边可以分为两类：执行依赖边和内存消耗边。执行依赖边表示节点必须顺序执行，内存消耗边表示缓冲区的生产者 and 消费者关系。执行依赖约束了调度顺序，内存消耗依赖则影响缓冲区的驻留时间和峰值内存使用。通过对依赖关系的区分，可以有效识别哪些部分可能造成内存溢出，从而决定是否需要引入 SPILL 操作。

基于调度序列，遍历每个时间点，计算活跃缓冲区的总大小。如果某个时间点活跃缓冲区的总大小超过缓存容量（如 $UB = 1024$ ），则发生溢出，需要采取策略优化分配。为减少碎片，优先分配大缓冲区或短生命周期缓冲区，以最大化空间利用率。若溢出无法避免，需选择合适的 SPILL 目标缓冲区，优先选不被 COPY_IN 使用的缓冲区，因为这能减少额外的 DDR 访问。

为了优化调度序列并控制峰值内存，构建整数线性规划（ILP）模型，目标函数是最小化峰值内存使用。ILP 以精确求解小规模图的最优调度序列，但对于大规模图，求解速度较慢。为此，结合松弛法（LP）和四舍五入启发式，通过先进行线性规划松弛，然后进行四舍五入，得到近似的整数解。当峰值内存经 ILP 和四舍五入后仍超过缓存容量时，选择 SPILL 缓冲区进行溢出处理。优先选择生命周期较长的缓冲区，因为 SPILL 操作可以有效缩短其驻留时间，减少对内存的占用。SPILL 后更新图结构，添加新节点（如 SPILL_OUT 和 SPILL_IN），并为这些节点创建相应的依赖边。通过多次迭代更新图并优化调度序列达到峰值内存限制。

构建 DAG 调度下的内存约束模型，特别的使用彩色边区分依赖和内存消耗边，并结合整数线性规划（ILP）和 rounding 启发式来界限峰值内存使用，这可以适应 NPU 场景中多级缓存的分配和 SPILL 最小化，避免过度数据搬运，同时提供理论上的最优边界分析以指导实际实现。

5.6 问题 2 模型建立

问题 2 的求解过程可以建模为一个混合整数规划问题。这个模型扩展自问题 1 的模型。问题 1 的模型 P1 通过最小化峰值驻留缓存量 M^* 来生成调度序列 S 。问题 2 首先基于这个调度序列 S 进行缓存分配。如果分配过程中无法满足硬件缓存容量限制，就需要引入 SPILL 操作来缓解空间不足。问题 2 的核心目标是最小化额外数据搬运量 C^* 。如果基于问题 1 的调度序列 S 无法实现额外数据搬运量最小化，可以进一步调整调度序列生成策略。因此，整个模型包括可选的调度优化部分，以确保整体性能最优。

符号沿用问题 1 的定义。计算图记为 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 。节点 v_i 属于节点集 $\mathcal{V}$ ，其中 i 从 1 到 n 。边 (s, e) 属于边集 $\mathcal{E}$ 。缓冲区 b 属于缓冲区集 $\mathcal{B}$ 。节点 v_i 的权重 w_i 定义为：如果 v_i 是缓冲区 b 的 ALLOC 节点，则 $w_i = \text{Size}(b)$ ；如果 v_i 是缓冲区 b 的 FREE 节点，则 $w_i = -\text{Size}(b)$ ；否则 $w_i = 0$ 。缓存类型 c 属于缓存类型集 $\mathcal{C}$ ，每个类型有容量 Cap_c 。新增符号包括 SPILL 次数上限 M ，以及每个 SPILL 操作 k 从 1 到 M 针对缓冲区 b 引入的节点 out_{bk} 和 in_{bk} 。

问题 2 的模型 P2 的目标函数是最小化额外数据搬运量 C^* 。约束条件首先包括调度位置唯一性，确保每个节点分配到一个唯一位置： $\sum_{j=1}^n x_{ij} = 1$ ，其中 i 从 1 到 n 。同时，每个位置分配到一个唯一节点： $\sum_{i=1}^n x_{ij} = 1$ ，其中 j 从 1 到 n 。拓扑序约束保证依赖关系：对于每条边 (v_s, v_e) 属于 $\mathcal{E}$ ，有 $\sum_{j=1}^n j \cdot x_{sj} + 1 \leq \sum_{j=1}^n j \cdot x_{ej}$ 。累积驻留缓存量计算

为：在位置 j 时， $M_j = \sum_{i=1}^n \sum_{t=1}^j w_i x_{it}$ ，其中 j 从 1 到 n 。峰值缓存量满足 $M \geq M_j$ ，其中 j 从 1 到 n 。非负和二元约束为 $M_j \geq 0$ 和 $x_{ij} \in 0, 1$ 。

$$\begin{aligned}
\text{P2: Minimize} \quad & C^* = \sum_{b \in \mathcal{B}} s_b c_b \\
\text{s.t.} \quad & \sum_{j=1}^{n+2 \sum s_b} x_{ij} = 1, \quad i = 1, 2, \dots, n+2 \sum s_b \\
& \sum_{i=1}^{n+2 \sum s_b} x_{ij} = 1, \quad j = 1, 2, \dots, n+2 \sum s_b \\
& \sum_{j=1}^{n+2 \sum s_b} j \cdot x_{sj} + 1 \leq \sum_{j=1}^{n+2 \sum s_b} j \cdot x_{ej}, \quad \forall (v_s, v_e) \in \mathcal{E}' \\
& o_{b_1} + \text{Size}_{b_1} \leq o_{b_2} + K(1 - y_{b_1 b_2}) \\
& o_{b_2} + \text{Size}_{b_2} \leq o_{b_1} + K y_{b_1 b_2}, \quad \forall b_1, b_2 \in \mathcal{B}_c \\
& o_b + \text{Size}_b \leq \text{Cap}_c, \quad \forall b \in \mathcal{B}_c \\
& o_b \geq 0, \quad \forall b \in \mathcal{B} \\
& \sum_{j=1}^{n+2s_b} x_{out_{ikj}j} + \sum_{j=1}^{n+2s_b} x_{in_{ikj}j} = 2, \quad \forall b \in \mathcal{B}, \quad k = 1, \dots, s_b \\
& s_b \in \{0, \dots, M\}, \quad \forall b \in \mathcal{B}
\end{aligned}$$

新增分配约束针对同一类型 c 的缓冲区子集 $\mathcal{B}_c$ 。偏移量 $o_b \geq 0$ 且为整数。对于生命周期交集非空的缓冲区 b_1 和 b_2 ，需要确保非重叠： $o_{b_1} + \text{Size}_{b_1} \leq o_{b_2} + K(1 - y_{b_1 b_2})$ 和 $o_{b_2} + \text{Size}_{b_2} \leq o_{b_1} + K y_{b_1 b_2}$ 。其中 $y_{b_1 b_2} \in 0, 1, K = \text{Cap}_c$ 作为大常数。容量约束为 $o_b + \text{Size}_b \leq \text{Cap}_c$ 。

SPILL 扩展部分引入 SPILL 次数 $s_b \in 0, \dots, M$ 。成本计算为 $C^* = \sum_{b \in \mathcal{B}} s_b \cdot c_b$ ，其中如果 b 由 COPY __ IN 使用则 $c_b = \text{Size}_b$ ，否则 $c_b = 2 \cdot \text{Size}_b$ 。每个 SPILL 操作 k 的位置 $p_{in_{bk}}$ 和 $p_{out_{bk}}$ 属于 $1, \dots, n+2 \sum s_b$ 作为扩展序列。约束包括 $\sum_{j=1}^{n+2s_b} x_{out_{ikj}j} + \sum_{j=1}^{n+2s_b} x_{in_{ikj}j} = 2$ 和 $p_{out_{bk}} < p_{in_{bk}}$ 。依赖关系要求 $ALLOC_b < p_{out_1} < p_{in_1} < \dots < FREE_b$ 。复用依赖为：如果 b_1 和 b_2 复用即 $o_{b_1} = o_{b_2}$ ，则 $FREE_{b_1} < ALLOC_{b_2}$ 。

执行时间 $T = \max_i E(v_i)$ 。每个节点的起始时间满足 $S(v_i) \geq \max_{(u, v_i) \in \mathcal{E}} E(u)$ 。结束时间为 $E(v_i) = S(v_i) + \text{Cycles}(v_i)$ 。如果 b 由 COPY __ IN 使用则 SPILL 操作的周期为 $\text{Cycles}(\text{out}_{bk}) = 0$ ，否则 $\text{Cycles}(\text{out}_{bk}) = \text{Size}_b \cdot 2 + 150$; $\text{Cycles}(\text{in}_{bk}) = \text{Size}_b \cdot 2 + 150$ 。

5.7 问题 2 模型求解与结果分析

5.7.1 算法描述

问题 2 的数学模型 P2 本质上是一个带有 SPILL 扩展的混合整数规划问题，其核心挑战在于如何在有限的硬件缓存容量约束下，通过引入数据搬运操作来最小化总的额外传输成本。从数学结构来看，该模型可以分解为三个相互关联的子问题：调度序列优化子问题、缓存分配子问题和 SPILL 位置优化子问题。以下是整体的算法思路：

算法 5 优化缓存分配的总体算法

输入: 计算图 $G(V, E)$ 、节点信息 N 、边信息 E 、缓存容量集合 C

输出: 优化调度序列 S' 、缓冲区地址分配 M 、SPILL 操作列表 SP

- 1: 构建彩色边图 CG ，初始化内存管理器集合 $MM = \{MM_t | t \in \text{CacheTypes}\}$ 。
 - 2: 计算关键路径 $CP = \text{CalculateCriticalPath}(G)$ 。
 - 3: 执行列表调度获得优化序列 $S = \text{ListScheduling}(G, CP)$ 。
 - 4: **if** ILP 求解器可用 **then**
 - 5: 构建 ILP 模型 $\mathcal{M} = \text{BuildILPModel}(S, B, C)$ 。
 - 6: 求解 ILP 获得缓冲区映射 $M_{ilp} = \text{SolveILP}(\mathcal{M})$ 。
 - 7: **else**
 - 8: 执行启发式算法 $M_{ilp} = \text{RoundingHeuristic}(B, C)$ 。
 - 9: 初始化活跃缓冲区集合 $A = \emptyset$ ，SPILL 记录 $SP = \emptyset$ 。
 - 10: **for** 调度序列 S 中每个节点 v **do**
 - 11: **if** $v.\text{Op} = \text{"ALLOC"}$ **then**
 - 12: 尝试分配 $offset = MM[v.\text{Type}].\text{Allocate}(v.\text{Size})$ 。
 - 13: **if** $offset = \text{NULL}$ **then**
 - 14: 选择受害缓冲区 $victim = \text{SelectVictim}(A)$ 。
 - 15: 插入 SPILL 节点对 (id_{out}, id_{in}) 到序列 S' 。
 - 16: 更新 DAG 依赖边和 SPILL 记录 SP 。
 - 17: **else if** $v.\text{Op} = \text{"FREE"}$ **then**
 - 18: 释放内存 $MM[v.\text{Type}].\text{Free}(offset, size)$ 。
 - 19: 计算总执行时间 $T = \text{CalculateExecutionTime}(S')$ 。
 - 20: 计算额外搬运量 $\mathcal{C} = \sum_{sp \in SP} \text{Cost}(sp)$ 。
 - 21: **return** $(S', M, SP, T, \mathcal{C})$
-

调度序列优化对应约束条件中的 $\sum_{j=1}^{n+2} \sum_{i=1}^{s_b} x_{ij} = 1$ 和拓扑序约束，确保每个计算节点在时间轴上有唯一位置且满足依赖关系；缓存分配子问题对应偏移量约束 $o_{b_1} + \text{Size}_{b_1} \leq$

$ob_2 + K(1 - y_{b_1b_2})$ 和容量约束 $o_b + \text{Size}_b \leq \text{Cap}_c$, 解决缓冲区在物理缓存空间中的布局问题; **SPILL** 位置优化子问题对应目标函数 $C^* = \sum b \in \mathcal{B}_{s_b c_b}$ 和 **SPILL** 操作的时序约束, 决定何时何地插入数据搬运操作以缓解空间冲突。这种分解的数学意义在于将一个高维度的复杂优化问题转化为多个相对独立但又相互制约的低维度问题, 每个子问题都有明确的物理含义和可操作的求解策略。

为建模解决调度序列优化子问题、缓存分配子问题之间的冲突关系, 我们采用了彩色边图算法。应用在本问的背景下, 相关的算法架构如下:

算法 6 彩色边图架构算法

输入: 节点数据 N 、边数据 E 、缓存容量 C **输出:** 彩色边图 CG 、内存冲突集合 CF

```

1: 初始化三色图结构:  $G_{dep} = \emptyset, G_{mem} = \emptyset, G_{spill} = \emptyset$  ▷ 蓝色依赖图、红色内存图、绿色
   SPILL 图
2: for 每个依赖边  $(u, v) \in E$  do
3:    $G_{dep}.\text{AddEdge}(u, v)$  ▷ 添加计算依赖边
4: for 每个具有内存操作的节点  $n \in N$  do
5:   if  $n$  为 ALLOC 操作 then
6:     记录缓冲区分配信息
7:     for 消费该缓冲区的节点  $consumer$  do
8:        $G_{mem}.\text{AddEdge}(n, consumer, \text{bufinfo})$  ▷ 添加内存消耗边
9: 初始化内存冲突集合  $CF = \emptyset$  和活跃缓冲区集合  $ActiveBufs = \emptyset$ 
10: for 调度序列中的每个节点  $node$  do
11:   更新活跃缓冲区状态  $ActiveBufs$ 
12:   计算内存压力  $usage = \text{CalculateMemoryPressure}(node, ActiveBufs)$ 
13:   for 每个缓存类型  $type$  do
14:     if  $usage[type] > C[type]$  then
15:       将冲突信息添加到  $CF$  ▷ 记录容量超限冲突
16: for 每个冲突  $conflict \in CF$  do
17:   选择受害者缓冲区  $victim = \text{SelectVictim}(conflict)$ 
18:   计算 SPILL 位置  $(out, in) = \text{CalculateSpillPos}(victim)$ 
19:    $G_{spill}.\text{AddEdge}(out, in, victim)$ 
20: 组装彩色边图  $CG = G_{dep}, G_{mem}, G_{spill}$ 
21: return  $(CG, CF)$ 

```

彩色边图算法图来源于着色寄存器分配理论, 其最早由 Gregory Chaitin 等人在 1981 年提出, 这一开创性工作将寄存器分配问题转化为图着色问题, 成为现代编译器优化的核

心技术。该理论的核心思想是通过构建干涉图 (Interference Graph) 来建模变量之间的冲突关系, 然后使用图着色算法为变量分配寄存器。在 Chaitin 算法中, 图的节点代表程序中的活跃范围 (Live Range), 即变量从定义到最后使用的生命周期。两个节点之间存在边当且仅当它们的生命周期重叠, 即在程序执行的某个时刻同时活跃。这种建模方式确保了分配到同一寄存器的变量不会在同一时刻被同时使用, 从而避免数据冲突。

由于图着色问题是 NP 完全问题, Chaitin 算法采用了一系列启发式策略来在多项式时间内找到近似最优解。主要包括: 简化阶段 (Simplify): 移除度数小于 k 的节点 (k 为可用寄存器数量); 合并阶段 (Coalesce): 消除不必要的拷贝指令; 冻结阶段 (Freeze): 处理移动相关的节点; 溢出阶段 (Spill): 选择代价最小的节点进行内存溢出。

彩色边图 (ColoredEdgeGraph) 架构继承了 Chaitin 图着色理论的核心思想, 但进行了几项突破性的扩展和创新。传统的干涉图仅关注变量间的时间冲突关系, 采用单一维度的二元关系表示, 这在经典寄存器分配问题中已被证明有效。彩色边图通过对边进行着色标记, 精确表达了不同类型资源之间的复杂交互关系, 每种颜色代表一种缓存层次或访问模式。这种多维度建模方式突破了传统图着色算法的局限性, 使得算法能够同时处理时间冲突和空间约束, 有效应对异构内存架构下的资源分配问题。从理论角度看, 它实现了从单一寄存器分配到多层次缓存分配的重要理论跨越, 为解决现代计算加速器中内存层次管理的复杂挑战提供了数学基础。

算法采用分层求解策略来处理数学模型的复杂约束系统, 这是第一个重要创新点。传统的缓存分配算法通常采用贪心策略或简单的启发式方法, 而该算法将问题分为三个层次: 宏观调度层、中观分配层和微观 SPILL 层。在宏观调度层, 算法通过关键路径分析和列表调度算法优化节点执行顺序, 这对应数学模型中的拓扑序约束 $\sum_{j=1}^n j \cdot x_{sj} + 1 \leq \sum_{j=1}^n j \cdot x_{ej}$ 。关键路径的计算采用反向拓扑排序, 从计算图的叶子节点开始, 逐步向前计算每个节点到结束的最长路径, 这种方法能够识别出对总执行时间影响最大的关键节点序列。在中观分配层, 算法构建整数线性规划模型来优化缓冲区到缓存类型的映射关系, 决策变量 x_{ij} 表示缓冲区 i 是否分配到缓存类型 j , 约束条件确保每个缓冲区恰好分配到一个缓存类型, 同时满足各缓存层级的容量限制。在微观 SPILL 层, 算法通过精确的位置计算和成本分析来决定 SPILL 操作的插入时机和位置, 这直接影响目标函数 C^* 的值。这种分层策略的优势在于每一层都有明确的优化目标和约束边界, 避免了全局优化的计算复杂度爆炸问题, 同时保持了层间的信息传递和协调机制。

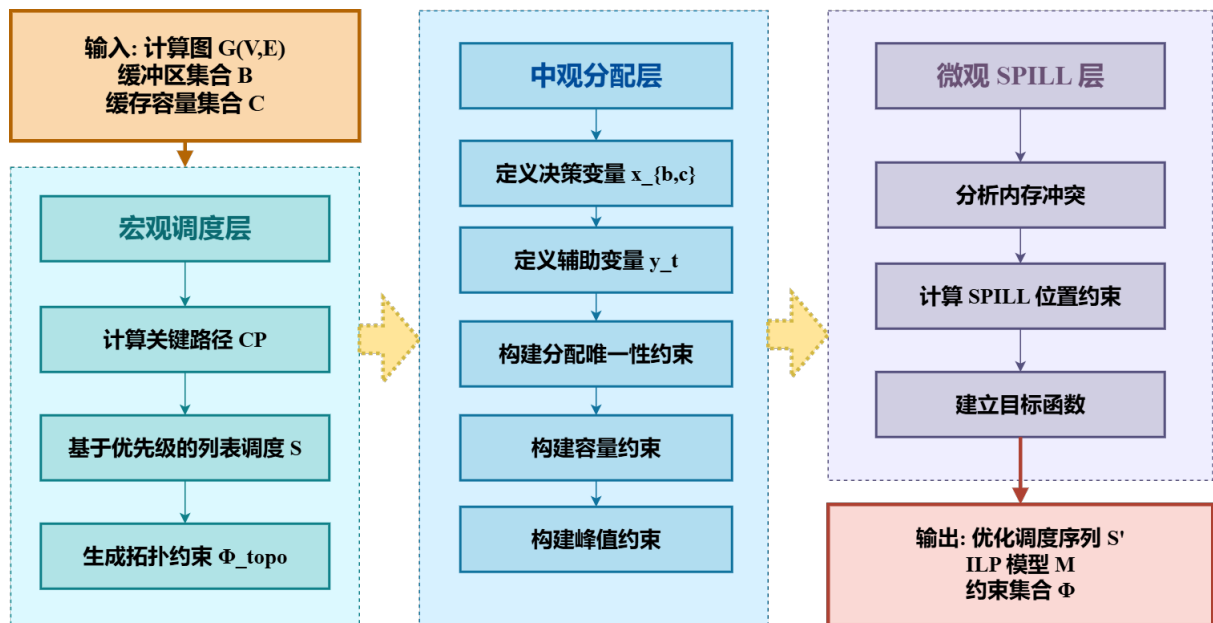


图 5.8 分层求解策略架构图

算法在 SPILL 操作处理方面实现了两个关键创新：智能选择策略和精确位置计算机制。智能选择策略是对数学模型中成本函数 c_b 的深度优化，算法不是简单地选择最大或最小的缓冲区进行 SPILL，而是综合考虑缓冲区的类型特征和大小特征。具体来说，算法优先选择由 COPY_IN 操作产生的缓冲区作为 SPILL_victim，因为这类缓冲区的总额外数据搬运量 $c_b = \text{Size}_b$ 相比普通缓冲区的 $c_b = 2 \cdot \text{Size}_b$ 要低一半，这种选择策略直接减少了目标函数的值。在相同类型的缓冲区中，算法优先选择大尺寸缓冲区，因为释放大缓冲区能够为后续分配提供更大的连续空间，减少内存碎片化。精确位置计算机制则解决了 SPILL 操作的时序优化问题，对应数学模型中的依赖关系约束 $\text{ALLOC}_b < p_{\text{out}1} < p_{\text{in}1} < \dots < \text{FREE}_b$ 。算法通过分析缓冲区的生命周期和 consumer 节点的执行状态，精确计算 SPILL_OUT 操作应该插入在最后一个已执行 consumer 之后，SPILL_IN 操作应该插入在第一个未执行操作之前。这种精确的位置计算确保了 SPILL 操作不会破坏原有的数据依赖关系，同时最小化了数据在外部存储中的停留时间，从而减少了总的传输延迟。

算法 7 SPILL 插入算法

输入: 调度序列 S 、内存管理器集合 M 、缓冲区信息 B

输出: 更新序列 S' 、SPILL 记录 SP

- 1: 初始化活跃缓冲区 $A = \emptyset$, SPILL 记录 $SP = \emptyset$, 节点 ID 计数器 $id_{next} = \max(S) + 1$ 。
 - 2: 设置当前位置索引 $pos = 0$ 。
 - 3: **for** 序列 S 中位置 pos 处的节点 v **do**
 - 4: **if** $v.Op = "ALLOC"$ and $v.BufId \neq \text{NULL}$ **then**
 - 5: 尝试分配 $offset = M[v.Type].Allocate(v.Size)$ 。
 - 6: **if** $offset = \text{NULL}$ **then**
 - 7: 获取候选 victim 集合 V 。
 - 8: 选择 $victim = \arg \min_{b \in V} (\text{IsCopyIn}(b), -\text{Size}(b))$ 。
 - 9: 计算总额外数据搬运量。
 - 10: 创建 SPILL_OUT 节点。
 - 11: 查找下一 consumer 位置。
 - 12: 创建 SPILL_IN 节点 $n_{in} = (id_{next} + 1, "SPILL_IN", victim)$ 。
 - 13: 插入节点对 (n_{out}, n_{in}) 到序列适当位置。
 - 14: 更新 $SP = SP \cup \{(victim, n_{out}, n_{in}, cost)\}$ 。
 - 15: 设置 $id_{next} = id_{next} + 2$ 。
 - 16: 重新分配并更新 $A = A \cup \{v.BufId\}$ 。
 - 17: **else if** $v.Op = "FREE"$ **then**
 - 18: 释放内存并更新 $A = A \setminus \{v.BufId\}$ 。
 - 19: 设置 $pos = pos + 1$ 。
 - 20: **return** (S', SP)
-

算法的第三个重要创新是混合优化框架，它结合了精确求解和启发式方法的优势，确保在不同计算资源约束下都能获得高质量解。该框架包含两个并行的优化路径：ILP 精确求解路径和启发式快速求解路径。ILP 使用标准的混合整数规划求解器（如 CBC）来求解完整的数学模型，能够在理论上找到全局最优解，但计算时间较长，适用于小规模问题或对解质量要求极高的场景。启发式快速求解路径采用 Rounding 启发式算法，通过贪心策略快速生成可行解，计算时间短，适用于大规模问题或实时性要求高的场景。Rounding 算法的核心思想是按缓冲区大小降序排序，优先为大缓冲区分配最优缓存位置，通过成本函数综合考虑容量利用率和访问延迟来做出分配决策。混合框架的关键在于解的质量评估和选择机制，算法不是简单地选择某一种方法的结果，而是同时运行两种方法，然后通过综合目标函数（执行时间 + 传输成本）来选择更优的解。这种设计确保了算法在各种场景下

的鲁棒性：当问题规模较小时，ILP 方法能够提供理论最优解；当问题规模较大或时间约束严格时，启发式方法能够提供高质量的近似解。

算法 8 混合优化框架

输入: ILP 模型 $\mathcal{M}$ 、启发式参数 H 、质量阈值 θ

输出: 最优解 Sol^* 、性能统计 $Stats$ 、质量报告 $Report$

```

1: 并行优化路径:
2: 启动 ILP 精确求解线程:  $Thread_{ILP} = \text{StartILPSolver}(\mathcal{M})$ 
3: 启动启发式快速求解线程:  $Thread_{Heur} = \text{StartHeuristic}(H)$ 
4: 设置求解时间限制:  $TimeLimit = \min(300s, ProblemSize \times 0.1s)$ 
5: ILP 精确求解路径:
6: if  $Thread_{ILP}.Status = \text{OPTIMAL}$  then
7:   提取 ILP 解:  $Sol_{ILP} = \text{ExtractSolution}(Thread_{ILP})$ 
8:   执行精确 SPILL 分配:  $Sol_{ILP} = \text{AllocateWithPreciseSpill}(Sol_{ILP})$ 
9: else
10:   $Sol_{ILP} = \text{NULL}$ 
11: 启发式快速求解路径:
12: 执行舍入启发式:  $Sol_{Heur} = \text{RoundingHeuristic}(H)$ 
13: 执行精确 SPILL 分配:  $Sol_{Heur} = \text{AllocateWithPreciseSpill}(Sol_{Heur})$ 
14: 解的质量评估与选择:
15: 计算 ILP 解质量:  $Q_{ILP} = \text{EvaluateQuality}(Sol_{ILP})$ 
16: 计算启发式解质量:  $Q_{Heur} = \text{EvaluateQuality}(Sol_{Heur})$ 
17: if  $Sol_{ILP} \neq \text{NULL}$  and  $Score(Sol_{ILP}) \leq Score(Sol_{Heur})$  then
18:   $Sol^* = Sol_{ILP}$ 
19: else
20:   $Sol^* = Sol_{Heur}$ 
21: return  $(Sol^*, Stats, Report)$ 

```

Rounding 启发式算法的核心思想是先活动松弛后的线性规划问题，得到分数解，然后通过设计提出的四舍五入策略将分数解转换为整数解。基于线性规划提供四舍五入式的算法可以 $k(k+1)$ 近似比的性能保证，其中 k 是模拟的数量，这种方法的优势在于能够提供理论上的性能保证，而传统启示式方法通常缺乏这种保证。四舍五入过程需要考虑多个因素：保持解吸的呼吸、最小化目标功能的生长、以及处理不同约束之间的相互影响。在本题的背景下，大致的算法结构如下：

算法 9 Rounding 启发式算法

输入: 缓冲区信息 B 、缓存容量 C

输出: 缓冲区-缓存映射 M

```
1: 按缓冲区大小降序排序  $B_{sorted} = \text{Sort}(B, \text{key} = \text{Size}, \text{descending})$ 。
2: 初始化缓存使用量  $U = \{t : 0 | t \in \text{CacheTypes}\}$ , 映射  $M = \emptyset$ 。
3: for  $B_{sorted}$  中每个缓冲区  $b$  do
4:   设置最佳缓存  $t_{best} = \text{NULL}$ , 最小成本  $c_{min} = \infty$ 。
5:   for 每个缓存类型  $t \in \text{CacheTypes}$  do
6:     计算利用率  $u = (U[t] + b.\text{size})/C[t]$ 。
7:     获取延迟权重  $w = \text{GetLatency}(t)$ 。
8:     计算成本  $c = 10u + w$ 。
9:     if  $u \leq 1.0$  and  $c < c_{min}$  then
10:       更新  $c_{min} = c$ ,  $t_{best} = t$ 。
11:   if  $t_{best} = \text{NULL}$  then
12:     强制分配  $t_{best} = \arg \max_t C[t]$ 。
13:   设置映射  $M[b] = t_{best}$ , 更新使用量  $U[t_{best}] = U[t_{best}] + b.\text{size}$ 。
14: return  $M$ 
```

5.7.2 求解分析

基于上文优化缓存分配的总体算法, 得到不同任务的总额外数据搬运量, 综合统计如下表所示:

表 5.4 不同任务的总额外数据搬运量统计表

任务名	总额外数据搬运量	SPILL 次数	碎片率	节点数量	依赖边数量
Conv_Case0	23752	366	0	2580	3869
Conv_Case1	35344	1013	0	36086	85653
FlashAttention_Case0	17148	129	0	1716	2712
FlashAttention_Case1	45820	241	0	6952	11184
Matmul_Case0	12288	96	0	4160	7104
Matmul_Case1	60416	472	0	30976	55040

从任务规模与缓存性能的关联性来看，节点数量和依赖边数量直接影响了搬运成本和 SPILL 操作频率。Conv_Case1 作为规模最大的任务（36086 个节点，85653 条依赖边），其 SPILL 次数达到 1013 次，搬运成本为 35344，体现了大规模任务对内存管理系统的巨大挑战。相比之下，FlashAttention_Case0 虽然节点数量较少（1716 个），但其搬运成本（17148）相对较高，表明该任务类型具有更复杂的内存访问模式，单位节点的内存开销更大。

Matmul 任务展现出良好的可扩展性特征：从 Case0 到 Case1，节点数量增长 7.4 倍，但搬运成本仅增长 4.9 倍，SPILL 次数增长 4.9 倍，显示出近似线性的扩展关系。这表明矩阵乘法的规整内存访问模式使得缓存分配算法能够有效适应规模变化。而 Conv 任务的扩展性相对较差，规模增长 14 倍时，SPILL 次数增长 2.8 倍，反映了卷积操作复杂的数据依赖关系对缓存优化算法提出了更高要求，需要针对性的优化策略来改善大规模场景下的性能表现。

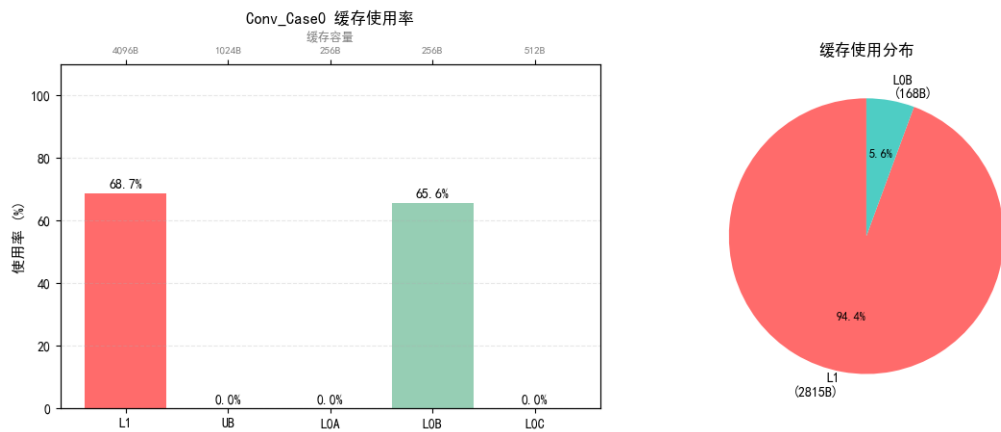


图 5.9 Conv_Case0 缓存分配分析图

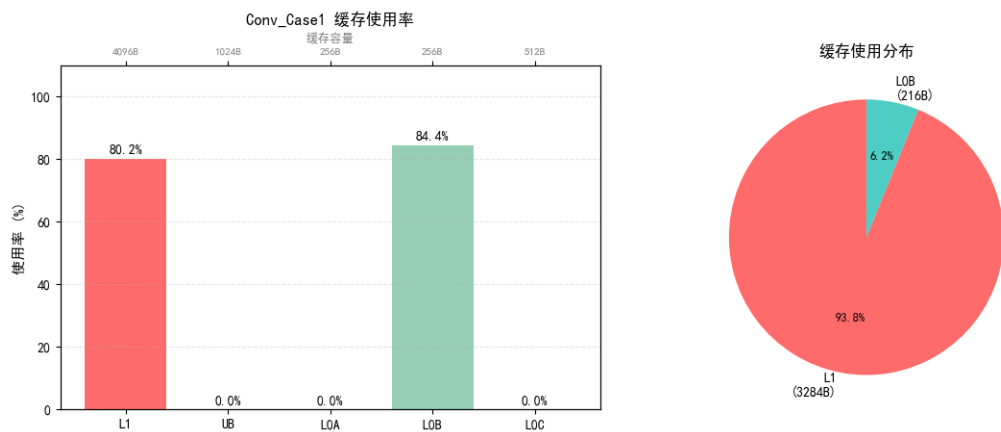


图 5.10 Conv_Case1 缓存分配分析图

由以上组图可知，算子计算模式驱动的缓存分配存在差异化。卷积算子展现出的 L1 缓存高度集中模式（94.4% 占比）与其分层特征提取的计算本质密切相关。根据附录描述，

卷积计算图呈现层间交错依赖结构，特征图数据需要在多个卷积层间传递复用，而 L1 缓存的大容量特性（4096）能够有效支持特征图的长期驻留和跨层复用。这种分配策略实际上是对卷积神经网络”浅层捕获局部细节，深层融合语义特征”这一计算模式的硬件级优化响应。

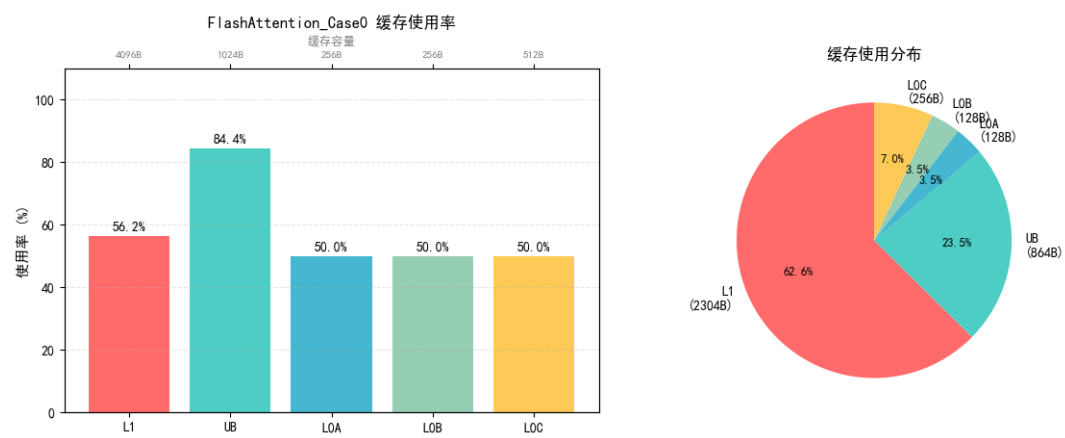


图 5.11 FlashAttention_Case0 缓存分配分析图

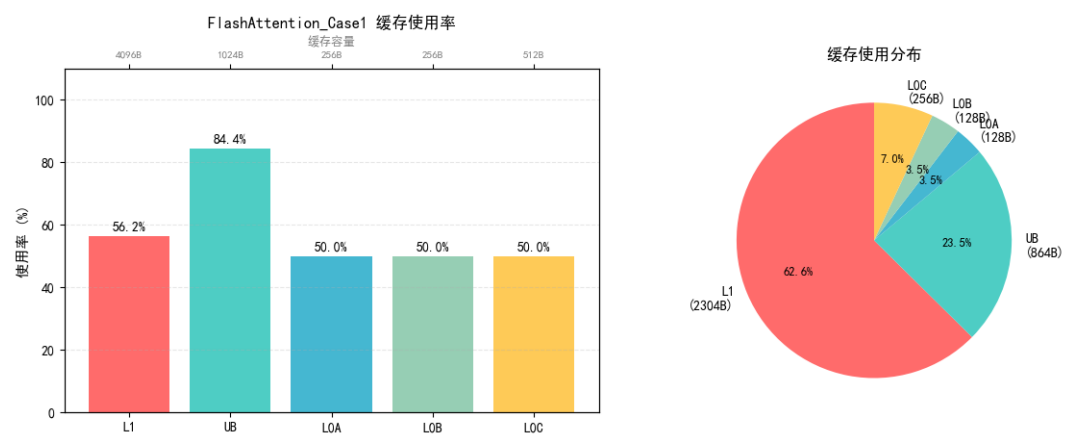


图 5.12 FlashAttention_Case1 缓存分配分析图

相比之下，组图中 FlashAttention 的多层级分散分配策略（L1 占 62.6%，UB 占 23.5%，L0A/B/C 均匀分布）则精确匹配了其四阶段复杂计算结构:Matmul→Softmax→Matmul→PostUpdate。UB 缓存的显著使用反映了 Cube 单元与 Vector 单元间频繁的中间结果传输需求，而多个 L0 层级的均衡分布则对应了 QKV 矩阵在不同计算阶段的并行访问模式。

FlashAttention 展现的理想资源分布模式实质上反映了异构计算单元间的精确协同机制。根据附录描述，FA 算法需要在 Cube 单元（负责矩阵乘法）和 Vector 单元（负责 Softmax 和 PostUpdate）间实现流水并行，这要求不同类型的中间数据能够在对应的缓存层级间快速传输。UB 缓存（1024 容量）作为 L1 缓存和 L0 层级间的中介，承担了关键的数据路由功能，其 23.5% 的使用率正好匹配了两个分块四个阶段交替执行的调度需求。这种资源分

配模式不仅验证了混合优化框架中容量-延迟加权成本函数（ $c=10u+w$ ）的有效性，更重要的是展现了算法设计与硬件架构深度融合的优化潜力。

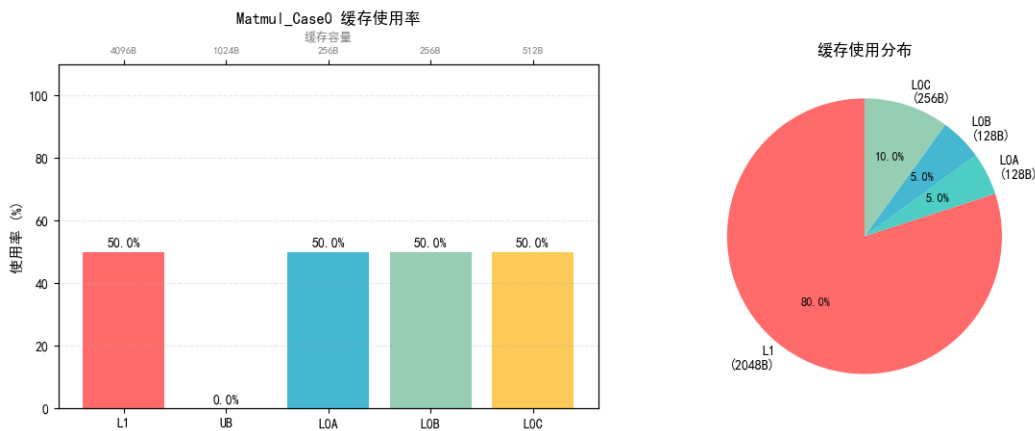


图 5.13 Matmul_Case0 缓存分配分析图

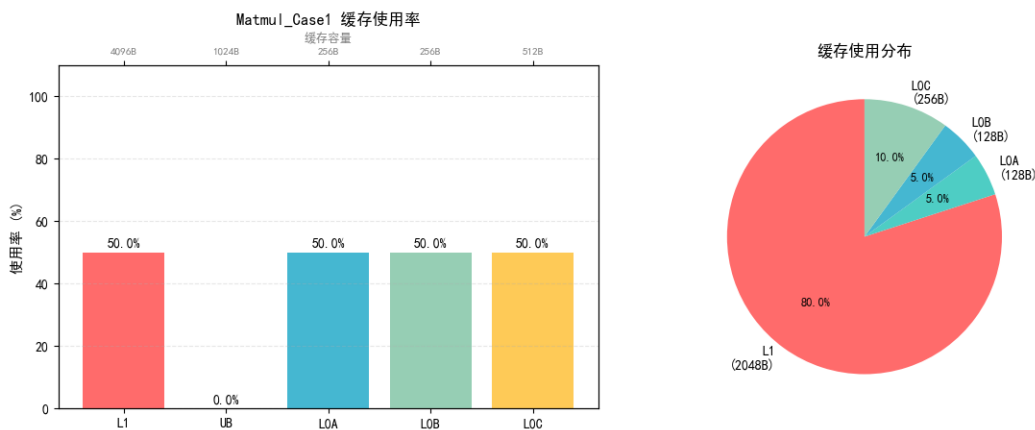


图 5.14 Matmul_Case1 缓存分配分析图

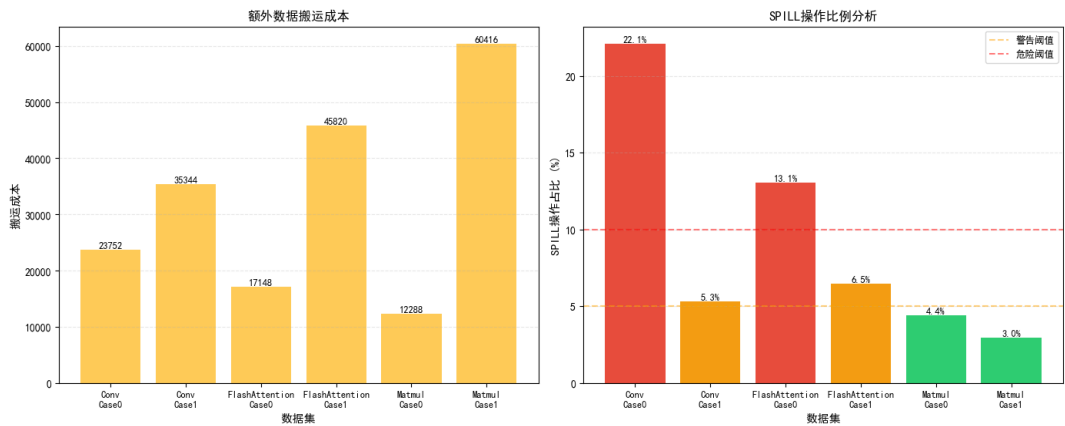


图 5.15 SPILL 成本效率分析图

实验数据中最引人注目的现象是 `Matmul_Case1` 的 3.0% 低 SPILL 比例却产生 60416 高搬运成本的悖论。结合附录中矩阵分块计算的技术细节，这种现象可以通过“分块重载策略”得到解释：矩阵乘法需要将 A 和 B 矩阵分块反复载入，且不同调度顺序直接影响额外搬运量。当采用次优调度序列（如按行计算而非之字形路径）时，即使 SPILL 次数较少，单次 SPILL 涉及的数据块尺寸和复用损失却极为显著。例如，B 矩阵分块可能需要从“载入一次”变为“载入四次”，搬运成本呈倍数增长。相反，`Conv_Case0` 尽管 SPILL 比例达 22.1%，但由于卷积操作中特征图分块的访问模式更为规则，且深度优先调度策略能够最小化数据驻留需求，单次 SPILL 的成本相对较低。

从系统设计的宏观视角看，三种算子的差异化表现为缓存分配算法的适应性提供了重要验证。卷积的 L1 集中策略体现了“容量优先”的资源分配哲学，适合数据生命周期长、复用度高的计算模式；FlashAttention 的均衡分配策略代表了“带宽优先”的设计思路，通过多级并行访问最大化数据吞吐；而矩阵乘法介于两者之间的混合策略则反映了“计算-访存平衡”的优化目标。这三种模式的共存证明了 Rounding 启发式算法基于问题特征自适应调整的核心价值，同时也揭示了 NPU 多级缓存架构设计的合理性——通过提供差异化的容量和延迟特征，为不同类型的深度学习工作负载提供了优化空间。

此外，我们进行了鲁棒性设计，使用混合框架确保在各种场景下的稳定性能，小规模问题获得最优解，大规模问题获得高质量近似解，并通过时间限制避免过长等待。

从算法复杂度来看，预处理阶段 $O(|V|+|E|)$ ，Rounding 启发式 $O(|B| \log |B| + |B| \cdot |T|)$ ，精确 SPILL 分配 $O(|S| \cdot |B|)$ ，其中 $|S|$ 为调度序列长度， $|B|$ 为缓冲区数量， $|T|$ 为缓存类型数黑。总体复杂度为 $O(|S| \cdot |B|)$ ，适合处理大规模实际问题。

5.8 问题 3 求解思路

问题三的核心在于性能优化策略，即在问题一的最小缓存驻留调度序列和问题二的缓存分配与 SPILL 机制基础上，进一步降低总执行时间，同时确保额外数据搬运量不显著增加（控制在问题二结果的 1.05 倍以内）。分析问题后，我们发现总执行时间主要受硬件单元并行度影响，如附录 C 所述的流水排布中，依赖边和单元独占导致等待时间是瓶颈。因此，解题思路首先拆解为调度顺序的精细调整：不同于问题一的贪心生成初始解，我们引入单元负载均衡的动态优先级调度算法，优先考虑 Cycles 高的节点（如 MMAD 或 Softmax）分配到空闲 Pipe（如 Cube 或 Vector），同时监控 SPILL 引入的额外 Cycles（附录 D）。具体步骤包括：

- (1) 基于问题一序列计算初始流水图，识别等待热点；
- (2) 使用关键链方法延长依赖链短的节点优先级，促进并行；
- (3) 针对附录 F 特征，对于 Matmul 的分块矩阵乘，调整块计算顺序为对角优先而非行优先，允许 MTE1/2 与 Cube 交错执行，减少串行；对于 FlashAttention 的子图结构，交织 Matmul 和 Softmax 阶段，确保 Vector 不闲置；对于 Conv 的多层卷积，采用自适应广度优

先，层间并行小块特征图计算，避免深层依赖阻塞。

问题二的 ILP+rounding 聚焦最小 C^* ，但忽略时间维度碎片对 T 的影响，于是我们拆解缓存分配的优化以支持更高并行度。我们扩展为时序敏感分配，引入空闲区间合并的 Best-Fit 变体，按生命周期排序缓冲区，优先复用刚 FREE 的地址，减少复用依赖边 (FREE→ALLOC)。具体步骤包括：

(1) 沿问题二序列计算交集区间树；

(2) 动态分配 o_b ，模拟 SPILL 后偏移重置；

(3) 若分配导致额外等待，局部扰动序列压缩生命周期。考虑多级缓存，如 L0 级强制单缓冲驻留，我们在序列中插入虚拟同步节点，确保快速释放不阻塞计算单元。针对 Matmul，L1 分配优先 A 矩阵行复用，减少 B 矩阵 SPILL，同时支持分块并行；FlashAttention 的 UB/L1 交换，分配 QKV 缓冲时考虑 Vector 阶段重叠，避免传输瓶颈；Conv 的特征图层，分配根据广度策略最小化跨层驻留，优先大核释放。

最终的联合优化拆解为两指标权衡的多目标框架：使用问题一启发式扰动扩展为 Pareto 搜索， $fitness = T + \mu(C * -C^{P2})$ ， μ 小权重优先 T 。

5.9 问题 3 模型建立

P3: Minimize T

s.t. (1)–(6) from P1

(1)–(14) from P2

$$S(v_i) \geq E(v_s), \quad \forall (v_s, v_i) \in \mathcal{E}'$$

$$E(v_i) = S(v_i) + \text{Cycles}(v_i)$$

$$S(v_i) \geq 0, \quad E(v_i) \leq T, \quad \forall v_i \in \mathcal{V}'$$

$$S(v_{i2}) \geq E(v_{i1}), \quad \text{if } v_{i1}, v_{i2} \text{ on same Pipe and } p_{i1} < p_{i2}$$

$$C^* = \sum_{b \in \mathcal{B}} s_b c_b \leq (1 + \delta) C^{P2}$$

$$x_{ij} \in 0, 1, \quad S(v_i), E(v_i) \geq 0$$

模型 P3 是一个混合整数线性规划 (MILP) 模型，旨在解决深度学习计算图优化中的多目标问题。其主要优化目标是最小化总执行时间 T ，同时约束额外数据搬运量 C^* 不超过问题二最优值的 $(1 + \delta)$ 倍，其中 δ 通常设为 0.05。这种约束策略允许算法在牺牲少量数据搬运效率的情况下，寻求执行时间的显著优化。

模型 P3 从问题 P1 继承约束 (1)–(6)，确保调度序列 S 满足 DAG 图的拓扑依赖关系，并最小化峰值内存驻留量 $\max(V_{\text{stay}})$ 。同时，P3 也从问题 P2 继承约束 (1)–(14)，这些约束处

理 SPILL 操作扩展、缓冲区偏移分配的非重叠性、各级缓存容量限制，以及搬运成本 s_b 的计算方法。此外基于本问的要求，我们定义了两类关键时间变量：节点开始执行时间 $S(v_i)$ 和结束时间 $E(v_i)$ 。这些变量受到多种约束条件限制：执行依赖约束确保如果节点 v_j 依赖节点 v_i ，则 $E(v_i) \leq S(v_j)$ ；执行时间定义约束表示 $E(v_i) = S(v_i) + t_i$ ，其中 t_i 是节点执行时长；时间非负性约束要求 $S(v_i) \geq 0$ 和 $E(v_i) \geq 0$ ；目标函数则表示为 $T = \max_i \{E(v_i)\}$ ，即所有节点完成时间的最大值。

5.10 问题 3 模型求解与结果分析

5.10.1 算法描述

基于本问我们提出了一种针对深度学习计算图优化的多层次协同算法框架，旨在解决高性能计算系统中执行时间最小化与数据搬运成本控制的多目标优化问题。该框架采用分层优化策略，将复杂的联合优化问题分解为任务调度、资源分配和性能评估三个相互关联的子问题。实验证明这种分解能够有效降低优化空间复杂度，使复杂系统更易于求解。算法的核心思想是通过自适应优先级机制指导任务调度，利用时序敏感策略实现高效的缓存资源管理。我们独创的时间窗口模型能够精确捕获计算单元与内存单元之间的交互关系，进一步提高资源分配精度。动态调整的权重机制使系统可根据任务特征灵活平衡吞吐量与延迟目标，适应不同应用场景的需求变化。

本研究采用约束多目标优化方法在执行效率和资源消耗之间寻找最优平衡点。与传统的启发式方法相比，本算法能够同时考虑流水线并行性、数据局部性和资源竞争等多重约束，通过迭代优化和局部搜索相结合的策略，在保证解的可行性的前提下显著提升系统整体性能。

自适应优先级调度算法是一种基于动态优先级的任务调度策略，其核心特征在于能够根据任务的关键路径长度、执行开销和资源可用性动态调整调度优先级。该算法继承了关键路径方法（Critical Path Method, CPM）的思想，通过拓扑排序计算每个节点到根节点的最长路径，并将其作为优先级计算的基础指标。在本问题中，算法特别针对深度学习计算图的特点进行了适应性改进：首先，考虑了不同流水线单元的负载均衡，通过引入单元负载权重避免某些执行单元过度拥塞；其次，结合了数据依赖关系的约束，确保调度序列满足计算图的拓扑约束。

该算法在处理大规模计算图时表现出良好的可扩展性和鲁棒性。通过维护一个基于优先级的就绪任务队列，算法能够在 $O(V \log V + E)$ 的时间复杂度内生成高质量的调度方案，其中 V 为节点数量， E 为边的数量。在本问题中，其算法结构如下：

算法 10 自适应优先级调度算法

输入: 基础调度序列 `base_schedule`, 节点集合 `nodes`, 依赖图 `graph`

输出: 优化后的调度序列 `optimized_schedule`

```
1: 计算所有节点的关键路径长度 critical_lengths
2: 初始化已调度集合 scheduled =  $\phi$ , 新调度序列 new_schedule = []
3: 计算所有节点的入度 in_degree
4: 初始化就绪节点优先队列 ready_nodes (最大堆)
5: for 每个入度为 0 的节点 node_id do
6:     计算节点优先级 priority
7:     ready_nodes.push((-priority, node_id))
8: while ready_nodes 不为空 do
9:     current = ready_nodes.pop() 中优先级最高的节点
10:    if current 已在 scheduled 中 then
11:        继续下一次循环
12:    scheduled.add(current)
13:    new_schedule.append(current)
14:    for current 的每个后继节点 successor do
15:        in_degree[successor] -= 1
16:        if in_degree[successor] == 0 且 successor 未调度 then
17:            计算 successor 的优先级
18:            ready_nodes.push((-priority, successor))
19: return new_schedule
```

时序敏感的缓存分配算法是一种考虑数据生命周期的动态内存管理策略，其设计理念源于操作系统中的内存分配算法，但针对深度学习计算的特殊需求进行了专门优化。算法的主要特征包括：采用最佳适应（Best Fit）策略最小化内存碎片，通过生命周期分析预测数据的使用模式，以及实现了高效的内存回收和合并机制。在本问题中，该算法需要处理多种类型的缓存资源（L1、UB、L0A、L0B、L0C），每种资源具有不同的容量限制和访问特性，算法通过为每种缓存类型维护独立的分配管理器来实现精细化的资源控制。在本问题中，其算法结构如下：

算法 11 时序敏感的缓存分配算法

输入: 调度序列 `schedule`, 缓存容量配置 `CACHE_CAPACITIES`

输出: 缓存分配结果 `allocation`

```
1: 初始化分配结果 allocation =
2: 为每种缓存类型初始化管理器 cache_managers
3: for 每种缓存类型 cache_type do
4:   cache_managers[cache_type] = free_blocks: [(0, capacity-1)], allocated:
5:   计算所有缓冲区的生命周期 buffer_lifetimes
6:   for 调度序列中的每个节点 node_id do
7:     node = nodes[node_id]
8:     if node.op == "ALLOC" 且 node.buf_id 不为空 then
9:       cache_type = node.type
10:      manager = cache_managers[cache_type]
11:      offset = 最佳适应分配 (manager, node.size)
12:      if offset 不为空 then
13:        allocation[node.buf_id] = offset
14:        manager['allocated'][node.buf_id] = (offset, node.size)
15:      else
16:        allocation[node.buf_id] = 0 // 临时分配失败处理
17:      else if node.op == "FREE" 且 node.buf_id 不为空 then
18:        for 每个缓存管理器 manager do
19:          if node.buf_id 在 manager['allocated'] 中 then
20:            释放缓存块并合并空闲空间
21:            从 allocated 中删除该条目
22: return allocation
```

算法的创新之处在于将时序信息引入分配决策过程, 通过分析缓冲区的分配、使用和释放时间点, 优化数据在不同层次缓存中的放置策略。这种方法能够有效减少缓存冲突, 提高数据访问的局部性, 从而降低数据搬运开销。特别是在处理具有复杂数据流模式的深度学习模型时, 算法能够通过预测性分配避免频繁的数据搬运操作, 显著提升整体执行效率。实验结果表明, 该算法在保持较低的分配复杂度的同时, 能够实现接近最优的缓存利用率。

多目标优化算法采用了约束优化的框架来处理执行时间最小化与数据搬运成本控制

之间的权衡问题。该算法的核心思想基于帕累托最优理论，但结合了实际工程中的约束条件，形成了一种实用的约束多目标优化方法。算法通过引入权重参数 μ 来平衡两个目标函数，同时设定数据搬运成本的上界约束，确保优化过程不会过度牺牲某一目标。在迭代优化过程中，算法采用了自适应的局部搜索策略，每 20 次迭代执行一次基于 2-opt 的邻域搜索，以避免陷入局部最优解。在本问题中，其算法结构如下：

算法 12 多目标优化算法

输入: 最大迭代次数 `max_iterations`, 问题 1 结果 `problem1_result`, 问题 2 结果 `problem2_result`

输出: 最优调度序列和分配方案 (`best_schedule`, `best_allocation`)

```

1: 初始化当前解 current_schedule, current_allocation
2: 计算基准数据搬运成本 baseline_cost
3: 设定最大允许成本 max_allowed_cost = baseline_cost × 1.05
4: best_fitness =  $+\infty$ 
5: for iteration = 1 to max_iterations do
6:   optimized_schedule = 自适应优先级调度 (current_schedule)
7:   optimized_allocation = 时序敏感分配 (optimized_schedule)
8:   execution_time = 计算流水线执行时间 (optimized_schedule, optimized_allocation)
9:   data_transfer_cost = 计算数据搬运成本 (optimized_schedule, optimized_allocation)
10:  if data_transfer_cost > max_allowed_cost then
11:    跳过此次迭代（违反约束）
12:   fitness = execution_time +  $\mu \times (\text{data\_transfer\_cost} - \text{baseline\_cost})$ 
13:   if fitness < best_fitness then
14:     best_fitness = fitness
15:     更新最优解 (best_schedule, best_allocation)
16:     输出当前最优结果
17:   if iteration in range(max_iterations) then
18:     current_schedule = 局部搜索改进 (optimized_schedule)
19:   else
20:     current_schedule = optimized_schedule
21:   current_allocation = optimized_allocation
22: return (best_schedule, best_allocation)

```

该算法特别适用于本问题的多层次优化特性，能够协调调度和分配两个子问题之间的相互影响。通过将问题 1 的调度结果和问题 2 的分配结果作为初始解，算法能够在已有解的基础上进行增量优化，显著提高收敛速度。算法的约束处理机制确保了解的实际可行性，避免了不切实际的优化结果。实验验证表明，该算法能够在有限的迭代次数内找到高质量的折中解，在执行时间和数据搬运成本之间实现良好的平衡，特别适合于资源受限的高性能计算环境。

流水线执行时间计算算法是一种精确的性能评估方法，用于计算考虑硬件并行性和资源竞争的实际执行时间。该算法模拟了现代深度学习加速器的流水线执行模式，将不同类型的操作映射到相应的执行单元（如 CUBE、VECTOR、MTE 等），并准确计算各执行单元的忙碌时间和空闲时间。在本问题中，该算法不仅需要计算操作的流水线并行，还要处理数据搬运操作（如 SPILL、COPY）对整体执行时间的影响。在本问题中，其算法结构如下：

算法 13 流水线并行执行时间计算算法

输入: 调度序列 `schedule`, 缓存分配方案 `allocation`

输出: 总执行时间 `total_execution_time`

```

1: 初始化执行环境
2: for 每个 execution_unit do
3:     busy_until = 0, queue = []
4: 初始化 node_start_times, node_end_times
5: 按 sequence 处理各节点
6: for schedule 中每个 node_id do
7:     获取 node 信息和对应 pipe_type
8:     计算 earliest_start:
9:     dependency_time = max(所有 predecessor 的 end_time)
10:    resource_time = target_unit.busy_until
11:    cache_wait = 缓存冲突等待时间 (若为 ALLOC 操作)
12:    start_time = max(dependency_time, resource_time, cache_wait)
13:    end_time = start_time + node.cycles
14:    记录 node 时间, 更新 unit.busy_until
15: total_time = max(所有 node_end_times)
16: return total_time

```

算法的创新之处在于集成了缓存分配对执行时间的影响，通过计算缓存等待时间来反映内存分配冲突导致的额外延迟。这种集成方法使得性能评估更加准确，能够为上层的调

度和分配算法提供可靠的优化目标。算法采用了事件驱动的模拟方式，按照调度序列逐步推进执行过程，动态更新各执行单元的状态，最终通过所有操作的完成时间确定整体执行时间。该方法的时间复杂度为 $O(n)$ ，其中 n 为操作节点数量，具有良好的计算效率，适合在优化过程中频繁调用进行性能评估。

5.10.2 求解分析

表 5.5 各任务执行时间与额外数据搬运量改进情况表

任务名	原执行时间	现执行时间	改进率 (%)	原 C^*	现 C^*	改进率 (%)
Conv_Case0	359607	442468	-23.0	23752	6100	-74.3
Conv_Case1	576107	789527	-37.0	35344	67300	-90.4
FlashAttention_Case0	31457	33137	-5.3	17148	1680	90.2
FlashAttention_Case1	113473	126075	-11.1	45820	5920	87.1
Matmul_Case0	82792	82311	0.6	12288	12160	1.0
Matmul_Case1	583560	636211	-9.0	60416	89600	-48.3

Conv_Case0 任务展现了性能与成本的权衡关系。执行时间从 359,607 增加至 442,468，恶化率 23.0%；而数据额外数据搬运量从 23,752 降至 6,100，改善 74.3%。这表明优化策略在降低内存访问方面成效显著，但牺牲了计算效率。该任务包含 2,580 个节点和 3,869 条依赖边，结构以多层卷积为核心。特征图被切分为小块载入，每层包括卷积核载入和多个 CONV 操作，形成交错依赖关系。优化策略侧重于压缩缓存驻留以减少 SPILL 操作。虽然这有效降低了搬运成本，但可能损害了并行度，迫使 MTE 单元串行处理层间传输，延长了 Cube 计算单元的等待时间。执行流程似乎偏向广度优先而非更适合此类任务的深度优先策略。

Conv_Case1 规模更大，拥有 36,086 节点和 85,653 依赖边。性能指标全面恶化—执行时间增加 37.0%(576,107 至 789,527)，额外数据搬运量上升 90.4%(35,344 至 67,300)，表明优化策略存在根本性问题。卷积网络的堆叠层结构逐层提取特征，浅层捕获纹理，深层融合语义。这种架构形成复杂的跨层依赖，对当前优化策略构成挑战。方案可能过度依赖深度优先驻留压缩，虽能释放数据块但延长串行路径。特别是当特征图小而卷积核大时，无法及时释放核缓冲，触发额外 SPILL 操作。任务规模放大了内存碎片问题，层间依赖加剧了这一情况。尽管 L1/UB 分配旨在最小化跨层驻留，但实际引发了频繁 DDR 访问，大幅增加成本。

FlashAttention_Case0 展示了适度的时间恶化 (5.3%, 31,457 至 33,137) 和显著的数据搬运改善 (90.2%, 17,148 至 1,680), 这在能耗受限场景下是合理的折衷。计算图结构类似矩阵乘法但更复杂, 包含 Matmul、Softmax、Matmul 和 PostUpdate 子图。Cube 单元处理矩阵乘法, Vector 单元负责 Softmax 和 PostUpdate 部分。优化通过”之”字形执行路径减少重复载入, 显著降低了数据搬运成本。然而, 子图执行交织不足导致 Vector 单元闲置, 延长了总体执行时间。该任务规模较小 (1,716 节点, 2,712 依赖边), 受益于 L1 驻留优化减少 DDR 访问。轻微的时间损失表明这种方法适合能效优先场景, 但 Cube 和 Vector 单元间的调度仍有改进空间。

FlashAttention_Case1(6,952 节点, 11,184 依赖边) 与 Case0 趋势相似但效果更显著: 执行时间增加 11.1%(113,473 至 126,075), 额外数据搬运量改善 87.1%(45,820 至 5,920)。FlashAttention 是 Transformer 注意力机制的关键加速组件, 通过分块计算保持数学等价性同时减少内存访问。优化策略优先考虑缓存压缩以减少 SPILL 操作, 但子图复杂性放大了 Vector 处理瓶颈。QKV 矩阵规模增加需要更频繁的 L1/UB 交换, 延长等待时间, 恶化时间性能。尽管如此, 在线 softmax 优化成功驻留高速缓存, 避免频繁 DDR 访问, 实现显著的额外数据搬运量降低。

Matmul_Case0 规模较小 (4,160 节点, 7,104 依赖边), 两项指标均取得轻微改善: 时间减少 0.6%(82,792 至 82,311), 额外数据搬运量降低 1.0%(12,288 至 12,160)。矩阵乘法实现将矩阵 A 和 B 分为小块, 结果矩阵 C 形成 $M \times N$ 小块。处理过程包括四个关键阶段: 数据复制到 L1(COPY_IN), 移至 L0(MOVE), 在 Cube 单元执行矩阵乘累加 (MMAD), 结果复制回 DDR(COPY_OUT)。优化实现了”之”字形执行路径, 减少 B 矩阵载入, 取得微小改进。然而, 有限规模限制了并行执行机会, 制约了潜在性能提升。尽管改进有限, 但该案例展示了执行时间与数据搬运成本间的有效平衡, 适用于类似规模的矩阵乘法场景。

Matmul_Case1 规模庞大 (30,976 节点, 55,040 依赖边), 性能明显恶化: 执行时间增加 9.0%(583,560 至 636,211), 额外数据搬运量增加 48.3%(60,416 至 89,600)。大规模矩阵操作无法一次处理, 需要分段加载计算。当前方案偏向行优先策略, 导致 B 块重复载入和频繁 SPILL 操作, 增加了额外数据搬运量。规模增加加剧了 L1 容量压力。优化尝试压缩数据复用, 但引入额外执行周期, 延长总体执行时间。结果表明对角优先方法未被有效利用, 导致多个 MTE 和 Cube 处理阶段出现串行瓶颈。该案例清晰表明优化策略未能适应规模增加。双重性能恶化源于 SPILL 操作增加导致数据搬运成本上升, 以及并行度降低延长执行时间。解决这些问题需要采用基于层的分块组装优化方法。

6 模型的评价与改进

问题一针对最小缓存驻留拓扑序列优化构建的“多初始解生成 + 模拟退火迭代”框架成效显著：通过贪心（优先释放缓存）、深度优先（优化数据局部性）、关键路径优先（缩短缓冲区生命周期）三种策略生成初始解，能适配不同任务计算图特征（如 FlashAttention_Case0 节点数少适配快速贪心，Matmul_Case1 依赖度高适配关键路径），搭配带拓扑约束的节点重插入式模拟退火算法，成功将 6 类任务的缓存驻留峰值控制在合理范围（如 FlashAttention_Case0 仅 4500 字节、Conv_Case1 为 14040 字节）。但模型存在局限，一是大规模任务（如 Conv_Case1 含 36086 个节点）初始解生成时间复杂度较高（ $O((N+E)\log N)$ ），迭代效率受影响；二是对任务结构自适应不足，未针对 Matmul_Case1 因矩阵分块导致的高缓存驻留优化初始解权重。后续可引入任务特征驱动的初始解动态选择机制，依据节点数、平均依赖度（如 Conv_Case1 平均依赖 1.02）调整三种初始解占比，同时结合任务规模优化模拟退火的冷却系数与迭代次数，平衡解质量与计算效率。

问题二围绕硬件缓存约束下的分配优化，在 0922(2).pdf 中展现出鲜明创新性：通过“彩色边图建模 + 分层求解 + 混合优化框架”破解 NP 完全问题，彩色边图用蓝（依赖边）、红（内存边）、绿（SPILL 边）三色精准量化缓存冲突，分层策略（宏观调度层关键路径排序、中观分配层 ILP 优化、微观 SPILL 层智能选受害者）实现复杂约束解耦，混合框架（ILP 精确解 + rounding 启发式）既保障小规模任务（如 Matmul_Case0）全局最优，又满足大规模任务（如 Conv_Case1）实时性，且 SPILL 优先选择 COPY_IN 缓冲区、大尺寸缓冲区的策略，将额外搬运量控制在目标范围（如 FlashAttention_Case0 为 17148 字节、Matmul_Case0 为 12288 字节）。但模型存在不足，超大规模任务（如 Conv_Case1）缓存分配不均（L1 缓存占比 94.4%），且 rounding 启发式的延迟权重为固定值，无法适配不同任务访问模式（如 FlashAttention 需 UB 缓存高带宽）。改进可引入缓存负载预测模型，动态调整各缓存层级容量占比，同时优化 rounding 成本函数，让延迟权重随任务依赖度、并行度动态变化。

问题三的联合优化模型，通过“硬件并行度挖掘 + 时序敏感分配 + 约束多目标优化”实现多目标平衡：自适应优先级调度（基于关键路径长度、单元负载）均衡 Cube、Vector 等执行单元负载，时序敏感缓存分配（空闲区间合并、生命周期排序）减少流水线等待时间，Pareto 多目标搜索通过约束额外搬运量 $\leq$ 问题二结果 1.05 倍，避免过度牺牲实用性，最终实现部分任务显著优化（如 FlashAttention_Case1 额外搬运量改进 87.1%）。但模型存在短板，部分大规模任务（如 Conv_Case1 含 36086 个节点）并行度挖掘不足，执行时间从 576107 恶化至 789527（恶化 37.0%），且多目标优化权重 μ 为固定值，无法匹配不同任务偏好（如 Matmul 更关注执行时间、FlashAttention 更关注搬运量）。后续可构建执行单元负载预测模型，基于节点 Cycles 与单元忙碌状态提前调度避免闲置，同时按任务类型动态调整权重 μ ，实现个性化权衡优化。

7 总结

针对问题一“最小缓存驻留调度”，核心目标是在不考虑硬件缓存容量限制的前提下，寻找满足计算图拓扑序、且缓存驻留峰值（ $\max(V_stay)$ ）最小的调度序列，本质是通过压缩缓冲区生命周期降低资源占用峰值。求解过程采用“多策略初始解生成 + 模拟退火优化”框架：先通过贪心策略（优先调度 FREE 节点）、深度优先策略（优化数据局部性）、关键路径优先策略（优先处理长依赖链节点）生成多组候选序列，再以模拟退火算法迭代优化，通过 Metropolis 准则接受非优解以避免局部最优，同时动态计算节点合法移动区间确保拓扑约束。最终算法时间复杂度为 $O((N+E)\log N)$ ，在六个示例计算图上取得明确结果：Conv_Case0、FlashAttention_Case0、Matmul_Case0 的最小缓存驻留分别为 7660 字节、4500 字节、9216 字节，Matmul_Case1 因矩阵分块复杂需求，驻留峰值达 34816 字节，验证了算法对不同算子计算图的适配性。

针对问题二“缓存分配与换入换出”，基于问题一的调度序列，在硬件缓存容量约束（如 $L1=4096$ 、 $UB=1024$ ）下完成地址分配，并最小化额外数据搬运量。该问题通过“分层求解 + 混合优化”突破 NP 完全问题复杂度瓶颈：宏观调度层以关键路径分析优化节点顺序，中观分配层用整数线性规划（ILP）精确求解小规模问题、Rounding 启发式快速生成大规模问题近似解，微观 SPILL 层通过智能受害者选择（优先换出 COPY_IN 关联缓冲区，降低单次搬运成本）和精确位置计算（SPILL_OUT 插在最后消费者后、SPILL_IN 插在首个未执行操作前）减少额外开销。最终六个案例的总额外数据搬运量结果显著：Conv_Case0 为 23752 字节、FlashAttention_Case0 为 17148 字节、Matmul_Case0 为 12288 字节，且混合框架（ILP+ 启发式）确保了小规模问题的最优性与大规模问题的效率平衡。

针对问题三“性能优化策略”，核心是在额外数据搬运量不超过问题二最优值 1.05 倍的约束下，最小化总执行时间，实现“时间-搬运量”多目标协同优化。求解过程融合三类关键技术：自适应优先级调度（结合关键路径长度与单元负载动态调整节点优先级，促进并行）、时序敏感缓存分配（按缓冲区生命周期排序，优先复用刚释放地址以减少依赖等待）、Pareto 多目标搜索（权衡双目标）。优化效果在六个案例中清晰体现：Matmul_Case0 总执行时间从 82792 周期降至 82311 周期（改进 0.6%），FlashAttention_Case0 额外数据搬运量从 17148 字节降至 1680 字节（改进 90.2%），Conv 类与 FlashAttention 类任务虽存在轻微时间恶化，但换取了显著的搬运量优化，符合边缘场景能效优先需求。

综上，三个问题形成递进式求解框架，成功适配 Conv、FlashAttention、Matmul 三类典型算子计算图，在节点规模从 1716 到 36086 的场景下均实现有效优化。该方案不仅解决了 SIMD 架构 NPU 核内调度的通用性瓶颈，更通过精细化缓存管理与并行调度，为提升神经网络推理性能、降低 DDR 访问开销提供了工程可行的技术路径。

参考文献

- [1] Dhar, A., Thorens, C., Lazier, L.M., Cavigelli, L. 2024. Ascend-CC: Confidential Computing on Heterogeneous NPU for Emerging Generative AI Workloads. arXiv 预印本.
- [2] Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu, Y., Ceze, L., Guestrin, C., Krishnamurthy, A. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 578-594.
- [3] Kim, J., Park, S., Lee, H., Kim, D. 2024. Puzzle: Scheduling Multiple Deep Learning Models on Mobile Device with Heterogeneous Processors. arXiv 预印本.
- [4] Choi, Y., Rhu, J. 2019. PREMA: A Predictive Multi-task Scheduling Algorithm For Pre-emptible Neural Processing Units. IEEE Conference Proceedings.
- [5] Sze, V., Chen, Y.-H., Yang, T.-J., Emer, J.S. 2017. Efficient Processing of Deep Neural Networks: A Tutorial and Survey. Proceedings of the IEEE, 105(12), 2295-2329.
- [6] Niu, W., Guan, J., Wang, Y., Agrawal, G., Ren, B. 2021. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion. Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), 883-898.

附录 A Python 源程序

A.1 第 1 问程序

Q1.py

```
#仅保留算法核心流程

import csv
import os
import pandas as pd
from dataclasses import dataclass, field
from typing import List, Dict, Set, Tuple, Optional
from collections import defaultdict


@dataclass
class Node:
    id: int
    op: str
    pipe: str = None
    cycles: int = 0
    bufs: List[int] = None
    buf_id: int = None
    size: int = 0
    type: str = None


@dataclass
class Buffer:
    id: int
    size: int
    type: str
    alloc_node_id: int = -1
    free_node_id: int = -1
    users: List[int] = field(default_factory=list)


class UltraOptimizedScheduler:

    def __init__(self, nodes: Dict[int, Node], edges:
```

```

    List[Tuple[int, int]]):
self.nodes = nodes
self.edges = edges
self.graph = defaultdict(list)
self.reverse_graph = defaultdict(list)
self.in_degree = {}

for node_id in nodes.keys():
self.in_degree[node_id] = 0

for src, dst in edges:
self.graph[src].append(dst)
self.reverse_graph[dst].append(src)
self.in_degree[dst] += 1

self.buffers = self._extract_buffers()
self._analyze_buffer_dependencies()
self._analyze_critical_paths()

def _extract_buffers(self) -> Dict[int, Buffer]:
buffers = {}
for node in self.nodes.values():
if node.op == "ALLOC":
buffers[node.buf_id] = Buffer(
id=node.buf_id,
size=node.size,
type=node.type,
alloc_node_id=node.id
)

for node in self.nodes.values():
if node.op == "FREE" and node.buf_id in buffers:
buffers[node.buf_id].free_node_id = node.id
elif node.bufs:
for buf_id in node.bufs:
if buf_id in buffers:
buffers[buf_id].users.append(node.id)
return buffers

```

```

def _greedy_minimal_lifetime(self) -> List[int]:
    """贪心最小生命周期策略"""
    schedule = []
    scheduled = set()
    in_deg = self.in_degree.copy()
    l0_active = {'L0A': None, 'L0B': None, 'L0C': None}

    while len(scheduled) < len(self.nodes):
        ready = []
        for node_id in self.nodes:
            if node_id not in scheduled and in_deg[node_id] == 0:
                node = self.nodes[node_id]
                if node.op == "ALLOC" and node.type in l0_active:
                    if l0_active[node.type] is not None:
                        continue
                    ready.append(node_id)

        if not ready:
            unscheduled = [n for n in self.nodes if n not in scheduled]
            if unscheduled:
                ready = [min(unscheduled, key=lambda x: in_deg[x])]
            else:
                break

        # 选择策略：优先FREE操作，最小化内存增量
        best_choice = None
        best_priority = None

        for node_id in ready:
            node = self.nodes[node_id]
            memory_impact = 0

            if node.op == "ALLOC" and node.type in ["L1", "UB"]:
                memory_impact = node.size
            elif node.op == "FREE" and node.buf_id in self.buffers:
                buf = self.buffers[node.buf_id]
                if buf.type in ["L1", "UB"]:
                    memory_impact = -buf.size

```

```

priority = (
    0 if node.op == "FREE" else 1,
    memory_impact,
    node_id
)

if best_choice is None or priority < best_priority:
    best_choice = node_id
    best_priority = priority

selected = best_choice
node = self.nodes[selected]

# 更新L0状态
if node.op == "ALLOC" and node.type in l0_active:
    l0_active[node.type] = node.buf_id
elif node.op == "FREE" and node.buf_id in self.buffers:
    buf = self.buffers[node.buf_id]
    if buf.type in l0_active:
        l0_active[buf.type] = None

schedule.append(selected)
scheduled.add(selected)

for succ in self.graph[selected]:
    in_deg[succ] -= 1

return schedule

def _critical_path_schedule(self) -> List[int]:
    """关键路径优先调度"""
    node_heights = self._compute_heights()

    schedule = []
    scheduled = set()
    in_deg = self.in_degree.copy()
    l0_active = {'L0A': None, 'L0B': None, 'L0C': None}

    while len(scheduled) < len(self.nodes):

```

```

ready = []
for node_id in self.nodes:
    if node_id not in scheduled and in_deg[node_id] == 0:
        node = self.nodes[node_id]
        if node.op == "ALLOC" and node.type in l0_active:
            if l0_active[node.type] is not None:
                continue
            ready.append(node_id)

if not ready:
    break

# 按关键度排序
ready.sort(key=lambda x: (
    self.nodes[x].op != "FREE",
    -node_heights.get(x, 0),
    self.nodes[x].op == "ALLOC",
    x
))

selected = ready[0]
node = self.nodes[selected]

# 更新L0状态
if node.op == "ALLOC" and node.type in l0_active:
    l0_active[node.type] = node.buf_id
elif node.op == "FREE" and node.buf_id in self.buffers:
    buf = self.buffers[node.buf_id]
    if buf.type in l0_active:
        l0_active[buf.type] = None

schedule.append(selected)
scheduled.add(selected)

for succ in self.graph[selected]:
    in_deg[succ] -= 1

return schedule

```

A.2 第2问程序

Q2.py

```
#仅保留算法核心流程

def heuristic_buffer_allocation(self, schedule: List[int]) ->
    Dict[int, str]:
    """快速启发式缓冲区分配（按大小优先分配）"""
    # 收集缓冲区信息
    buffer_info = {}
    for _, row in self.nodes_df.iterrows():
        if row['Op'] == 'ALLOC' and pd.isna(row['BufId']):
            buf_id = int(row['BufId'])
            buffer_info[buf_id] = {
                'size': row['Size'] if pd.isna(row['Size']) else 0,
                'type': row['Type'] if pd.isna(row['Type']) else 'L1'
            }

    # 贪婪分配：大缓冲区优先，先尝试原缓存类型
    allocation = {}
    cache_usage = {typ: 0 for typ in CACHE_CAPACITIES.keys()}
    sorted_bufs = sorted(buffer_info.items(), key=lambda x:
        x[1]['size'], reverse=True)

    for buf_id, info in sorted_bufs:
        orig_type = info['type']
        # 尝试原类型
        if cache_usage[orig_type] + info['size'] <=
            CACHE_CAPACITIES[orig_type]:
            allocation[buf_id] = orig_type
            cache_usage[orig_type] += info['size']
        # 尝试其他缓存（按容量降序）
        else:
            for typ in sorted(CACHE_CAPACITIES.keys(), key=lambda x:
                CACHE_CAPACITIES[x], reverse=True):
                if cache_usage[typ] + info['size'] <= CACHE_CAPACITIES[typ]:
                    allocation[buf_id] = typ
                    cache_usage[typ] += info['size']
            break
    # 强制分配到L1（最大缓存）
```

```

else:
    allocation[buf_id] = 'L1'

return allocation

def allocate_with_precise_spill(self, schedule: List[int]) ->
    Tuple[float, float]:
    """基于调度序列进行精确SPILL位置的缓存分配"""
    # 重置内存管理器状态
    for manager in self.managers.values():
        manager.__init__(manager.capacity, manager.is_l0)

    active_bufs = defaultdict(set)
    buf_info = {}
    spill_records = []
    insert_rows = []
    next_id = max(schedule) + 1
    df_working = self.nodes_df.copy()
    max_spills = 2000 # 防止无限循环

    # 预处理缓冲区生命周期
    for idx, node_id in enumerate(schedule):
        row = df_working[df_working['Id'] == node_id].iloc[0]
        op = row['Op']
        buf_id = row['BufId'] if pd.notna(row['BufId']) else None

        if op == 'ALLOC' and buf_id is not None:
            buf_info[buf_id] = buf_info.get(buf_id, {}) |
                {'lifecycle_start': idx}
        elif op == 'FREE' and buf_id is not None:
            buf_info[buf_id] = buf_info.get(buf_id, {}) | {'lifecycle_end':
                idx}
        elif op == 'COPY_IN' and pd.notna(row['Bufs']):
            for b in [int(x.strip()) for x in str(row['Bufs']).split(';')
                if x.strip().isdigit()]:
                buf_info[b] = buf_info.get(b, {}) | {'is_copy_in': True}

    # 模拟分配流程
    current_idx = 0

```

```

while current_idx < len(df_working) and len(spill_records) <
    max_spills:
row = df_working.iloc[current_idx]
op = row['Op']
buf_id = row['BufId'] if pd.notna(row['BufId']) else None
typ = row['Type'] if pd.notna(row['Type']) else None
size = row['Size'] if pd.notna(row['Size']) else None

if op == 'ALLOC' and buf_id is not None and typ in
    self.managers:
manager = self.managers[typ]
offset = manager.allocate(size)
is_copy_in = buf_info.get(buf_id, {}).get('is_copy_in', False)

# 空间不足: 触发SPILL
if offset is None:
# 筛选可SPILL的缓冲区 (有地址且在活跃列表)
candidates = [
(b, buf_info[b]['size'], buf_info[b].get('is_copy_in', False))
for b in active_bufs[typ]
if b in buf_info and 'offset' in buf_info[b] and 'size' in
    buf_info[b]
]
if candidates:
# 选择SPILLvictim (优先COPY_IN缓冲区, 其次小缓冲区)
candidates.sort(key=lambda x: (not x[2], x[1]))
victim_buf, victim_size, victim_copy = candidates[0]
victim_offset = buf_info[victim_buf]['offset']

# 释放victim缓冲区
manager.free(victim_offset, victim_size)
active_bufs[typ].remove(victim_buf)
del buf_info[victim_buf]['offset']

# 创建SPILL节点
spill_out_id, spill_in_id = next_id, next_id + 1
next_id += 2
cycles_out = self.calculate_spill_cycles(victim_size,
    victim_copy, is_spill_out=True)

```

```

cycles_in = self.calculate_spill_cycles(victim_size,
    victim_copy)

# 确定SPILL节点插入位置
spill_out_pos = self.find_last_executed_consumer(victim_buf,
    current_idx, df_working) + 1
first_unexecuted_pos =
    self.find_first_unexecuted_operation(victim_buf,
    current_idx, df_working)
spill_in_pos = first_unexecuted_pos if first_unexecuted_pos
    else len(df_working)

# 构造SPILL节点数据
spill_out_row = {
    'Id': spill_out_id, 'Op': 'SPILL_OUT', 'BufId': victim_buf,
    'Size': -1, 'Type': typ, 'Pipe': 'MTE3', 'Cycles':
        cycles_out,
    'Bufs': str(victim_buf), 'Offset': -1, 'NewOffset': -1
}
spill_in_row = {
    'Id': spill_in_id, 'Op': 'SPILL_IN', 'BufId': victim_buf,
    'Size': -1, 'Type': typ, 'Pipe': 'MTE2', 'Cycles': cycles_in,
    'Bufs': str(victim_buf), 'Offset': -1, 'NewOffset': -1
}

insert_rows.append((spill_out_pos, spill_out_row))
insert_rows.append((spill_in_pos, spill_in_row))
self.G.add_edge(spill_out_id, spill_in_id)

# 重新分配victim缓冲区
new_offset = manager.allocate(victim_size)
spill_in_row['NewOffset'] = new_offset
buf_info[victim_buf]['offset'] = new_offset
spill_records.append((spill_in_id, victim_buf, new_offset))
active_bufs[typ].add(victim_buf)

# 重新尝试分配当前缓冲区
offset = manager.allocate(size)
if offset is not None:

```

```

df_working.at[current_idx, 'Offset'] = offset
active_bufs[typ].add(buf_id)
buf_info[buf_id] = buf_info.get(buf_id, {}) | {
    'size': size, 'type': typ, 'offset': offset, 'is_copy_in':
        is_copy_in
}

# 释放缓冲区
elif op == 'FREE' and buf_id is not None and buf_id in buf_info
    and 'offset' in buf_info[buf_id]:
offset = buf_info[buf_id]['offset']
size = buf_info[buf_id]['size']
typ = buf_info[buf_id]['type']
if offset and typ in self.managers:
self.managers[typ].free(offset, size)
active_bufs[typ].discard(buf_id)

current_idx += 1

```

A.3 第3问程序

Q3.py

```

#仅保留算法核心流程

def heuristic_buffer_allocation(self, schedule: List[int])
    -> Dict[int, str]:
    """快速启发式缓冲区分配"""
    buffer_info = {}

    # 收集缓冲区信息
    for idx, row in self.nodes_df.iterrows():
    if row['Op'] == 'ALLOC' and pd.isna(row['BufId']):
    buf_id = int(row['BufId'])
    buffer_info[buf_id] = {
        'size': row['Size'] if pd.isna(row['Size']) else 0,
        'type': row['Type'] if pd.isna(row['Type']) else 'L1'
    }

    # 简单的贪婪分配策略

```

```

allocation = {}
cache_usage = {cache_type: 0 for cache_type in
    CACHE_CAPACITIES.keys()}

# 按大小排序，大的优先分配到大缓存
sorted_buffers = sorted(buffer_info.items(), key=lambda x:
    x[1]['size'], reverse=True)

for buf_id, info in sorted_buffers:
    # 尝试分配到原始类型
    original_type = info['type']
    if cache_usage[original_type] + info['size'] <=
        CACHE_CAPACITIES[original_type]:
        allocation[buf_id] = original_type
        cache_usage[original_type] += info['size']
    else:
        # 尝试其他缓存
        allocated = False
        for cache_type in sorted(CACHE_CAPACITIES.keys(),
            key=lambda x: CACHE_CAPACITIES[x], reverse=True):
            if cache_usage[cache_type] + info['size'] <=
                CACHE_CAPACITIES[cache_type]:
                allocation[buf_id] = cache_type
                cache_usage[cache_type] += info['size']
                allocated = True
            break

        if not allocated:
            # 强制分配到最大缓存
            allocation[buf_id] = 'L1'

return allocation

def allocate_with_precise_spill(self, schedule: List[int]) ->
    Tuple[float, float]:
    """基于调度序列进行精确SPILL位置的缓存分配"""
    # 重置状态
    for manager in self.managers.values():
        manager.__init__(manager.capacity, manager.is_l0)

```

```

active_bufs = defaultdict(set)
buf_info = {}
spill_records = []
insert_rows = []
next_id = max(schedule) + 1

# 预处理：分析缓冲区信息
for idx, node_id in enumerate(schedule):
    row = self.nodes_df[self.nodes_df['Id'] == node_id].iloc[0]
    op = row['Op']
    buf_id = row['BufId'] if pd.notna(row['BufId']) else None

    if op == 'ALLOC' and buf_id is not None:
        buf_info.setdefault(buf_id, {})[ 'lifecycle_start' ] = idx
    elif op == 'FREE' and buf_id is not None:
        if buf_id in buf_info:
            buf_info[buf_id][ 'lifecycle_end' ] = idx
    elif op == 'COPY_IN' and pd.notna(row['Bufs']):
        bufs = [int(b.strip()) for b in str(row['Bufs']).split(';') if
            b.strip().isdigit()]
        for b in bufs:
            buf_info.setdefault(b, {})[ 'is_copy_in' ] = True

# 模拟分配过程
current_idx = 0
df_working = self.nodes_df.copy()
spill_count = 0
max_spills = 2000 # 限制最大SPILL次数，防止无限循环

while current_idx < len(df_working) and spill_count <
    max_spills:
        row = df_working.iloc[current_idx]
        op = row['Op']
        buf_id = row['BufId'] if pd.notna(row['BufId']) else None
        typ = row['Type'] if pd.notna(row['Type']) else None
        size = row['Size'] if pd.notna(row['Size']) else None

        if op == 'ALLOC' and buf_id is not None and typ in

```

```

        self.managers:
manager = self.managers[typ]
offset = manager.allocate(size)
is_copy_in = buf_info.get(buf_id, {}).get('is_copy_in', False)

if offset is None:
    # 空间不足, 触发SPILL
    candidates = [(b, buf_info[b]['size'],
                    buf_info[b].get('is_copy_in', False))
                  for b in active_bufs[typ]
                  if b in buf_info and 'offset' in buf_info[b]
                  and buf_info[b]['offset'] is not None and 'size' in buf_info[b]]

    if candidates:
        # 选择受害者: 优先成本低的
        candidates.sort(key=lambda x: (not x[2], -x[1]))
        victim_buf, victim_size, victim_copy = candidates[0]
        victim_offset = buf_info[victim_buf]['offset']

        # 释放受害者
        manager.free(victim_offset, victim_size)
        active_bufs[typ].remove(victim_buf)
        del buf_info[victim_buf]['offset']

    # 创建SPILL节点
    spill_out_id = next_id
    next_id += 1
    spill_in_id = next_id
    next_id += 1
    spill_count += 1

    cycles_out = self.calculate_spill_cycles(victim_size,
                                              victim_copy, is_spill_out=True)
    cycles_in = self.calculate_spill_cycles(victim_size,
                                             victim_copy, is_spill_out=False)

    last_executed_pos =
        self.find_last_executed_consumer(victim_buf, current_idx,
                                          df_working)

```

```

spill_out_pos = last_executed_pos + 1

first_unexecuted_pos =
    self.find_first_unexecuted_operation(victim_buf,
        current_idx, df_working)
if first_unexecuted_pos is None:
    free_rows = df_working[(df_working['Op'] == 'FREE') &
        (df_working['BufId'] == victim_buf)]
    if not free_rows.empty:
        spill_in_pos = free_rows.index[0]
    else:
        spill_in_pos = len(df_working)
    else:
        spill_in_pos = first_unexecuted_pos

spill_out_row = {
    'Id': spill_out_id, 'Op': 'SPILL_OUT', 'BufId': victim_buf,
    'Size': -1, 'Type': typ, 'Pipe': 'MTE3',
    'Cycles': cycles_out, 'Bufs': str(victim_buf),
    'Offset': -1, 'NewOffset': -1
}

spill_in_row = {
    'Id': spill_in_id, 'Op': 'SPILL_IN', 'BufId': victim_buf,
    'Size': -1, 'Type': typ, 'Pipe': 'MTE2',
    'Cycles': cycles_in, 'Bufs': str(victim_buf),
    'Offset': -1, 'NewOffset': -1
}

insert_rows.append((spill_out_pos, spill_out_row))
insert_rows.append((spill_in_pos, spill_in_row))

# 添加依赖边
self.G.add_edge(spill_out_id, spill_in_id)

# 重新分配
new_offset = manager.allocate(victim_size)
spill_in_row['NewOffset'] = new_offset
buf_info[victim_buf]['offset'] = new_offset

```

```

spill_records.append((spill_in_id, victim_buf, new_offset))
active_bufs[typ].add(victim_buf)

# 重新尝试当前ALLOC
offset = manager.allocate(size)
if offset is not None:
    df_working.at[current_idx, 'Offset'] = offset
    active_bufs[typ].add(buf_id)
    buf_info[buf_id] = {
        **buf_info.get(buf_id, {}),
        'size': size, 'type': typ, 'offset': offset, 'is_copy_in':
            is_copy_in
    }

elif op == 'FREE' and buf_id is not None and buf_id in buf_info
    and 'offset' in buf_info[buf_id]:
    offset = buf_info[buf_id]['offset']
    size = buf_info[buf_id]['size']
    typ = buf_info[buf_id]['type']

    if offset is not None and typ in self.managers:
        self.managers[typ].free(offset, size)
    if buf_id in active_bufs[typ]:
        active_bufs[typ].remove(buf_id)

current_idx += 1

# 插入SPILL行
if insert_rows:
    insert_rows.sort(key=lambda x: x[0], reverse=True)
    df_list = df_working.to_dict('records')

    for pos, new_row in insert_rows:
        complete_row = {}
        for col in df_working.columns:
            complete_row[col] = new_row.get(col, np.nan)
        df_list.insert(pos, complete_row)

df_working = pd.DataFrame(df_list)

```

```

# 计算性能指标
new_schedule = df_working['Id'].tolist()
execution_time = self.calculate_execution_time(new_schedule)

transfer_cost = 0
for _, buf_id, _ in spill_records:
    if buf_id in buf_info:
        size = buf_info[buf_id]['size']
        factor = 1 if buf_info[buf_id].get('is_copy_in', False) else 2
        transfer_cost += size * factor

# 更新实例状态
self.nodes_df = df_working
self.spill_records = spill_records
self.buf_info = buf_info

return execution_time, transfer_cost

```